

A knowledge-informed optimization framework for performance-based generative design of sustainable buildings

Zhaoji Wu^a, Zhe Wang^{a,b,*}, Jack C.P. Cheng^{a,*}, Helen H.L. Kwok^c

^a Department of Civil and Environmental Engineering, The Hong Kong University of Science and Technology, Hong Kong, China

^b HKUST Shenzhen-Hong Kong Collaborative Innovation Research Institute, Futian, Shenzhen, China

^c Institute for the Environment, The Hong Kong University of Science and Technology, Hong Kong, China

HIGHLIGHTS

- A knowledge graph for performance-based generative design (PGD-KG) is proposed.
- Automated PGD-KG generation algorithm from parametric design models is developed.
- Reasoning for automated compliance checking & performance evaluation is developed.
- The PGD-KG optimization framework narrows the solution space of optimization.
- The PGD-KG optimization framework saves computational time by 73.25%.

ARTICLE INFO

Keywords:

Generative design
Knowledge graph
Sustainable building
Optimization
Building performance

ABSTRACT

Long computational time poses a significant obstacle to the practical utilization of performance-based generative design (PGD) in the early design stage. This study proposes a knowledge-informed PGD optimization framework for sustainable buildings, aimed to mitigate the time issue by integrating a knowledge graph (KG) into PGD. The first component of the framework is a PGD-KG schema that represents the topological relations within PGD. A generation method is then proposed for automatically developing PGD-KG models from parametric design models that are enhanced with semantic information. Furthermore, cross-domain reasoning algorithms are developed to enable automated compliance checking and performance evaluation based on regulatory requirements and sustainable design strategies, respectively. The proposed framework is applied to a design project focused on optimizing module layout to minimize cooling energy and maximize daylighting. The results demonstrate that the proposed framework can generate a satisfactory number of Pareto-optimal solutions while reducing computational time by 73.25% compared with the general optimization framework.

1. Introduction

Buildings exert a substantial influence on the environment, significantly contributing to worldwide energy consumption, greenhouse gas emissions, and resource depletion. Specifically, in 2021, buildings were responsible for 34% of total final energy consumption and 37% of global energy and process emissions. [1]. The necessity to address the environmental repercussions of the built environment has propelled a notable surge in the demand for sustainable buildings. This growing momentum stems from the pressing need to mitigate such impacts effectively. According to the MacLeamy curve [2], it is less expensive to make a change in the earlier stages of a project. Consequently,

integrating sustainable performance considerations early in the design process of buildings is crucial [3].

Early design is a complex task that involves multiple factors, e.g., building programs, regulatory requirements, client expectations, and the environmental impacts resulting from the built environment [4]. Building shapes have significant impacts on sustainable performance and various parameters related to building shapes should be focused on during the early design stage [5]. Performance-based generative design (PGD) is a powerful generative method that helps mitigate the complexity of early design by using computational methods to generate design cases in various parameters and shapes while integrating sustainable performance considerations. It takes into account the

* Corresponding author at: Department of Civil and Environmental Engineering, The Hong Kong University of Science and Technology, Hong Kong, China.

E-mail addresses: cezhewang@ust.hk (Z. Wang), cejcheng@ust.hk (J.C.P. Cheng).

<https://doi.org/10.1016/j.apenergy.2024.123318>

Received 22 December 2023; Received in revised form 20 April 2024; Accepted 24 April 2024

Available online 9 May 2024

0306-2619/© 2024 Elsevier Ltd. All rights reserved.

optimization of building performance by considering factors such as energy use, daylighting, thermal comfort, structural performance, etc. [4]. The current PGD framework relies on rule-based form generation, extensive performance evaluations through large-scale simulations and metaheuristic algorithms for optimizing designs, and the computational requirements of the current PGD framework are quite demanding in nature [6]. Wang et al. [7] demonstrated that it took 70 h to identify the optimal design, which is unacceptable for early design stages, where prompt conceptualization, modeling, and feedback are crucial. Therefore, reducing the computational time to an acceptable level is the urgent demand of PGD of sustainable buildings, and a new PGD framework that can generate optimized design scenarios within a reasonable computational timeframe is needed.

Several studies [8,9] have emphasized the importance of knowledge-driven generative design processes. For example, design knowledge from experts or architects could help minimize the size of the solution space [8] and customize the optimization process [9]. Various domain knowledge of sustainable building design is accumulated in the existing literature (such as [10–12]), which provides ample knowledge sources to be utilized in PGD of sustainable buildings. However, no study has yet explored the integration of knowledge into PGD. A knowledge graph (KG) presents knowledge as a set of concepts within specific domains as well as the relations among the concepts [13], and could hence serve as a powerful tool for knowledge-driven generative design processes, including PGD, offering advantages such as interoperability, cross-domain linking, and logical inference [14].

However, significant questions must be answered to integrate a KG into PGD: How can we effectively capture and represent the topological relations and functionalities inherent in PGD using KGs? How can we generate KG models specifically tailored for PGD from design models enriched with semantic information? Furthermore, how can we leverage these KG models to enhance the optimization framework of PGD and minimize computational resource requirements?

To address these critical issues, this study proposes a knowledge-informed optimization framework for PGD of sustainable buildings. The first component of the framework is a PGD-KG schema that represents the topological relations within PGD. A generation method is then proposed for automatically developing PGD-KG models from parametric design models that are enhanced with semantic information. Further, cross-domain reasoning algorithms are developed to enable automated compliance checking and performance evaluation based on regulatory requirements and sustainable design strategies, respectively. The proposed framework is applied to a design project focused on optimizing module layout to minimize cooling energy and maximize daylighting. The primary contribution of this study lies in the integration of human domain knowledge, particularly sustainable design strategies, into the optimization procedure of PGD.

The following sections in this paper are organized as follows. In Section 2, previous studies related to the PGD optimization framework and knowledge representation in the early design stage are presented and analyzed. In Section 3, the innovative knowledge-informed optimization framework is proposed. In Section 4, the proposed framework is implemented in a design project to illustrate how the framework works practically. Experiments are conducted to evaluate the performance of the framework, and results and discussion are given. In Section 5, the main conclusions and findings of this study are summarized.

2. Literature review

2.1. PGD optimization framework

Generative design employs computational techniques and algorithms to produce a diverse array of design options [15], and PGD sets building performance as the optimization objective that guides the direction of the generation [4]. Many studies have utilized PGD as a powerful tool to generate building design schemes towards

sustainability goals. For example, Lakhdari et al. [16] took a middle school classroom in a hot and dry region as their case for study. The window-to-wall ratio, glazing type, and wall type were optimized to improve daylighting and thermal comfort while reducing the annual energy loads. Saryazdi et al. [17] optimized the building envelopes, internal gains, and air conditioning systems of a residential building using an artificial neural network as the surrogate simulation model and a multi-objective genetic algorithm as the optimization algorithm. Significant reductions in energy consumption, hours of discomfort, and carbon emissions were achieved, by 39.3%, 62.8%, and 40.5% respectively, compared with the base case.

The current PGD framework is based on rule-based form generation, large-scale simulations for performance evaluation, and metaheuristic algorithm-based searches for optimized designs. The nature of the current framework leads to a substantial demand for computational power, and the computational time represents a significant challenge impeding the advancement of PGD [18]. Table 1 lists studies on PGD that have reported the computational time of their PGD optimization. The computational time ranged from 6 h to 81 h, which is unacceptable for early design stages. Thus, some studies have focused on improving the optimization framework to reduce the computational time. For example, Su and Yan [9] proposed an offline simulation with a divide and conquer (D&C) method in which similar simulation results can be reused, while Ramallo-González and Coley [19] developed sequential optimization by dividing the optimization step into coarse evaluation and precise evaluation steps. Wang et al. [7] introduced the hierarchical structuring of design variables to avoid infeasible solutions and thereby reduce redundant simulations, and Choudhary et al. [20] conducted hierarchical structuring of optimization objectives by passing the design variables to the next layer for optimizing sub-targets once the current layer of objectives was satisfied. These studies made efforts to enhance the PGD optimization framework but did not deviate from the computational nature that characterizes the framework, i.e., demanding a substantial computational power to conduct form generation, performance simulations, and search for optimized designs. For example, a large number of simulations were still involved, and it took approximately 70 h for each optimization run in Wang et al.'s study [7].

The long computational time primarily stems from the simulation programs used. Thus, surrogate models have been proposed to alleviate the issue of high computational demand. The idea behind surrogate modeling is to emulate an expensive high-fidelity model, namely a building simulation model, using a statistical model [21]. The typical process of developing a surrogate model includes problem definition, base model implementation, database generation, model fitting, and model validation [21]. However, the database generation step is still time-consuming. For example, Magnier and Haghighat [22] took three weeks to generate the database underlying the surrogate model, which indicates that the computational time is still an issue to be solved in using surrogate models in the PGD framework. Consequently, an innovative PGD framework that can alleviate the computation time is still needed.

2.2. Knowledge representation in the early design stage

Researchers have emphasized the importance of knowledge-driven generative design processes. Machairas et al. (2014) [8] stated that “the best tools and algorithms for the building design problem are the ones that enable researchers to use their expert knowledge to minimize the size of the solution space or drive the search to the right direction” (p. 105). Meanwhile, Su and Yan (2015) [9] claimed that “we should utilize architects' design knowledge to customize the optimization process, a process that would significantly save in computing time and eventually make optimization practical for architectural design” (p. 467). Although some researchers have recognized the importance of design knowledge, no study has yet explored the integration of knowledge into PGD.

Table 1

Summary of studies on PGD reporting computational time.

Study	Year	Building	Decision variable	Objective	Optimization algorithm	Computational time
[23]	2005	A single-story office building	Building orientation Aspect ratio Window type WWR Wall type and layer Roof type and layer	Life cycle cost Life cycle environmental impact	MOGA	39 h in total
[24]	2015	A single-story residential building	Window size and glazing material	Annual energy use Daylighting	NSGA-II	56 h in total
[25]	2016	A high-rise building	Building shape (with control points)	Structural displacement Construction cost Visual perception	NSGA-II	81 h in total
[26]	2020	A high-rise building	Building shape (with control points)	Structural analysis	RBFOpt CMA-ES	Simulation using CFD: 45 h in total Simulation using FFD: 6 h in total
[27]	2020	Rural tourism buildings	Building shape WWR	Energy consumption Daylighting Thermal comfort performance	SPEA-2	1.5 h for one simulation and 1 d for one generation
[28]	2023	High-rise office buildings	Building shape (with control points) Building height Photovoltaic panel Interaction height Wind-turbine interaction height	EUI Solar energy potential Wind energy potential Structural displacement	RBFMOpt NSGA-II	RBFMOpt: 22 h in total NSGA-II: 48 h in total
[29]	2023	An office building	Building shape (including an atrium) WWR Building orientation Shade	Thermal comfort Daylighting Solar radiation	SPEA-2 HypE	28.65 h in total
[30]	2023	Two residential multifamily buildings	Building typology Building orientation	Heating demand Cooling demand	NSGA-II	23.75 h in total

* Acronyms in this table: CFD: Computational Fluid Dynamic; CMA-ES: Covariance Matrix Adaptation Evolution Strategy; FFD: Fast Fluid Dynamic; HypE: Hyper-volume Estimation Algorithm for Multi-objective Optimization; MOGA: Multi-Objective Genetic Algorithm; NSGA-II: Non-Dominated Sorting Genetic Algorithm II; RBFOpt: Radial Basis Function Optimization; RBFMOpt: Radial Basis Function Multi-objective Optimization; SPEA-2: Strength Pareto Evolutionary Algorithm-2; WWR: Window-to-Wall Ratio.

Knowledge representation is a domain-specific issue, and various KG models have been developed to capture domain-specific knowledge related to buildings. KG models that could be applied to early design stages include ifcOWL, Building Topology Ontology (BOT), Brick Schema, and the SimModel ontology [31], as listed in Table 2. ifcOWL serves as an ontological representation of Industry Foundation Classes (IFCs), which are essentially a translation of IFC data in EXPRESS format [32]. BOT is a minimal ontology model that describes the core topological concepts within a building and focuses on topological relations among building elements [33]. Brick Schema provides detailed descriptions of physical, logical and virtual assets in buildings and the relations between them (especially heating, ventilation, and air conditioning (HVAC) systems) [34], and the SimModel ontology focuses on data integration for building energy simulations and offers an interoperable solution for connecting building simulation data with other building information [35]. Currently, however, no domain-specific KG model is available for PGD during the early design stages. Although the aforementioned models could be repurposed to assist in knowledge representation for PGD, there is a lack of study on how to leverage the expert knowledge (e.g., sustainable design strategies) efficiently and

automatically, and how to utilize the KG models to improve the PGD optimization process.

In summary, computational time is a major challenge that prevents the implementation of PGD in the early design stage. To address this challenge, there is a need for an innovative PGD framework that can mitigate the computational time constraints. While researchers have suggested that incorporating knowledge could enhance the PGD framework, no studies have been conducted on the integration of knowledge into PGD. The objective of this study is to provide a knowledge-informed optimization framework for PGD of sustainable buildings. The specific contributions of this study are as follows: 1) we propose a KG schema, referred to as PGD-KG, to effectively capture and represent the topological relations and functionalities within PGD; 2) we propose a generation method to automatically develop PGD-KG models from parametric design models that have been enriched with semantic information; 3) we propose a set of algorithms for automated compliance checking and building models evaluation based on cross-domain reasoning using the PGD-KG models; and 4) we evaluate the performance of the knowledge-informed optimization framework in terms of accuracy and computational time.

Table 2

Summary of KG models that could be applied to early design stages.

Name	Year	Scope
ifcOWL [32]	2016	Comprehensive description of geometric and semantic entities and properties of buildings and building-related components.
BOT [33]	2019	Topological relations among building elements.
Brick Schema [34]	2016	Physical, logical and virtual assets in buildings and the relations between them.
SimModel [35]	2014	Data integration and exchange of building energy simulations.

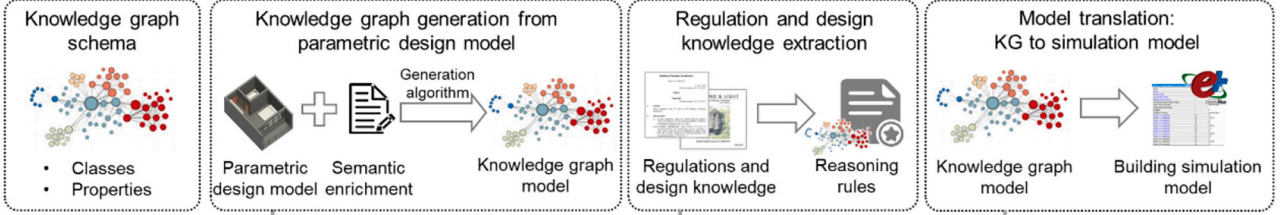
3. Methodology

3.1. Knowledge-informed optimization framework

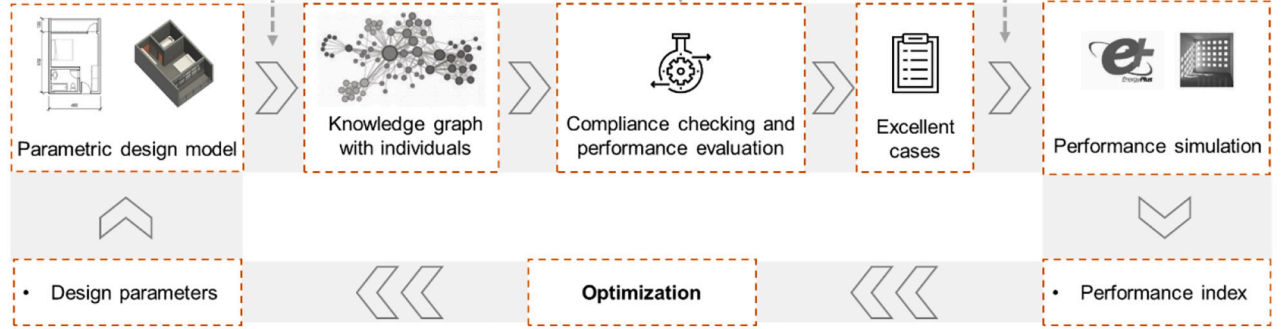
To introduce knowledge to the PGD framework seamlessly, a KG model for PGD and its applications are proposed. Firstly, a KG schema, named PGD-KG, is proposed to capture and represent the topological

relations and functionalities in PGD. A detailed description of the PGD-KG schema is given in Section 3.2. Secondly, a generation method that develops the KG models within the PGD-KG schema is proposed. A detailed description of the generation method is shown in Section 3.3. Thirdly, cross-domain reasoning algorithms for automated compliance checking and performance evaluation are proposed. Detailed descriptions of the reasoning algorithms are given in Section 3.4 and

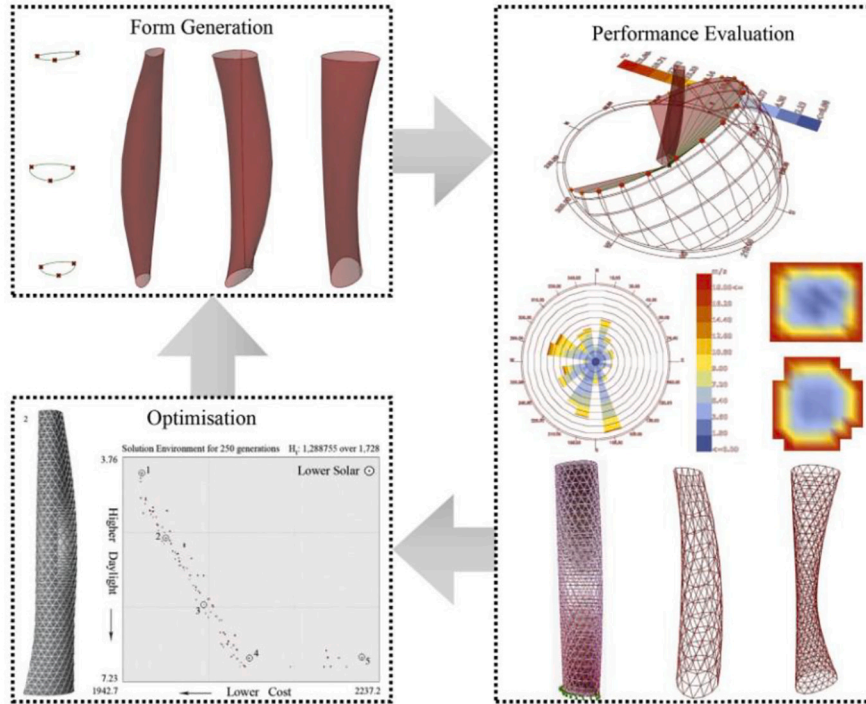
Knowledge graph for performance-based generative design



Knowledge-informed optimization framework



(a) Overview of the knowledge-informed optimization framework for PGD of sustainable buildings



(b) General PGD optimization framework [4]

Fig. 1. (a) Overview of the knowledge-informed optimization framework for PGD of sustainable buildings. (b) The general PGD optimization framework for comparison.

Section 3.5 respectively. The final step is to translate the KG model into the building simulation model, which is not the focus of this study. Details about the methods that translate KG models into building simulation models (e.g., building energy models) can be found in [36].

Fig. 1 (a) shows the overview of the knowledge-informed optimization framework for the PGD of sustainable buildings. Different to the general PGD optimization framework (Fig. 1 (b)) [4], knowledge is integrated into the performance evaluation part of the proposed framework.

Fig. 2 illustrates the detailed workflow of the knowledge-informed optimization framework, wherein each iteration represents a case with a distinct set of design parameters. Firstly, a parametric design model is developed by designers according to the design parameters and form generation rules, which is generally the first step in PGD. Secondly, the PGD-KG model is generated based on the generation method (Section 3.3) from the parametric design model with enriched semantic

information. Thirdly, the reasoning process for compliance checking (Section 3.4) is conducted. If the case cannot meet all regulatory requirements, it will be discarded, and a new iteration begins. Fourthly, for the cases that pass the compliance check, performance evaluation (Section 3.5) is conducted to check whether the case meets all the sustainable design targets under a given threshold, a user-decided hyper-parameter to determine the selection scale of cases. A case that satisfies all the targets is subsequently translated into the simulation model. Otherwise, the case is discarded, and a new iteration begins. Finally, a performance simulation is conducted, and performance metrics are obtained. Meta-heuristic optimization algorithms (e.g., Genetic Algorithm (GA) [37] and Particle Swarm Optimization (PSO) [38]) are used to optimize the design parameters according to the performance metrics, as in the general PGD framework.

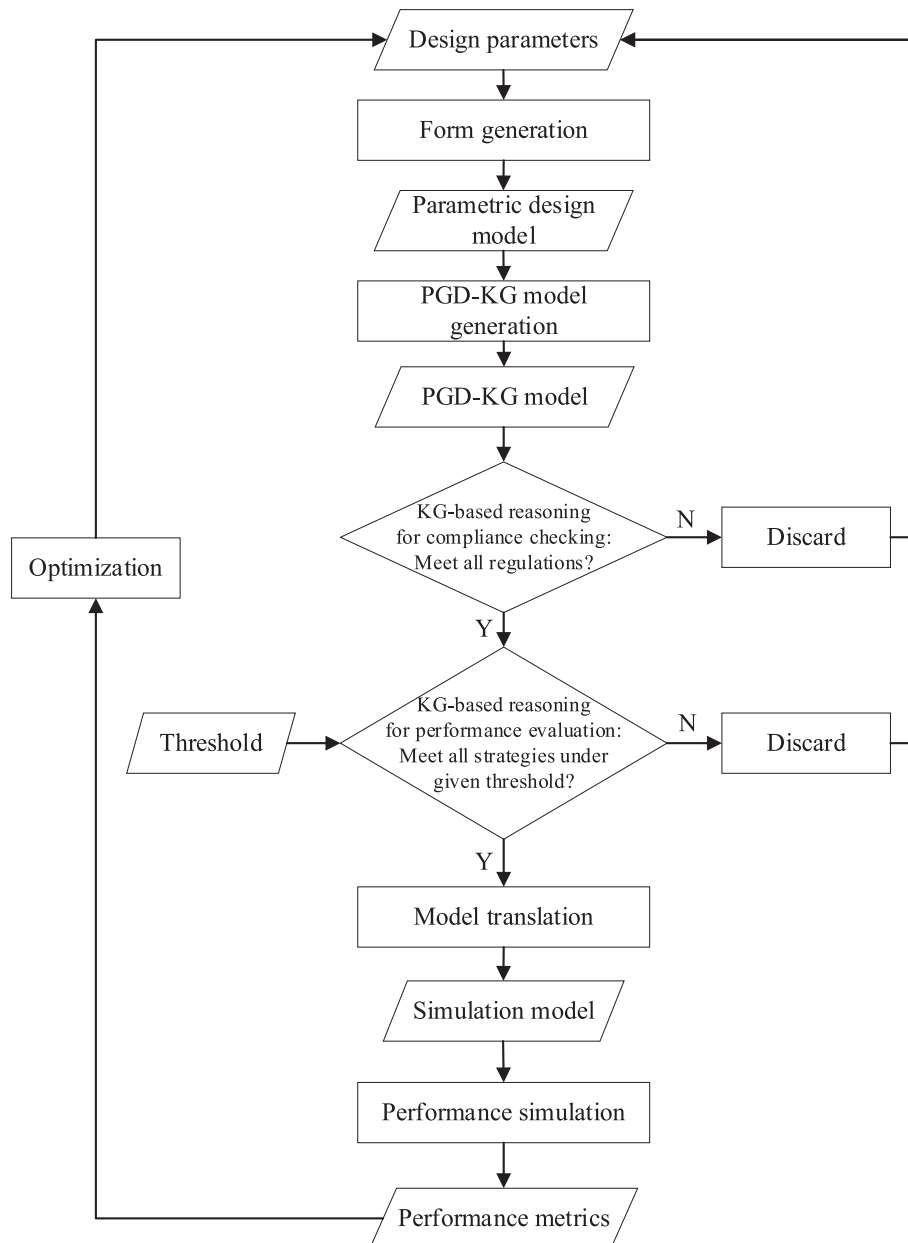


Fig. 2. Flowchart of the knowledge-informed optimization framework.

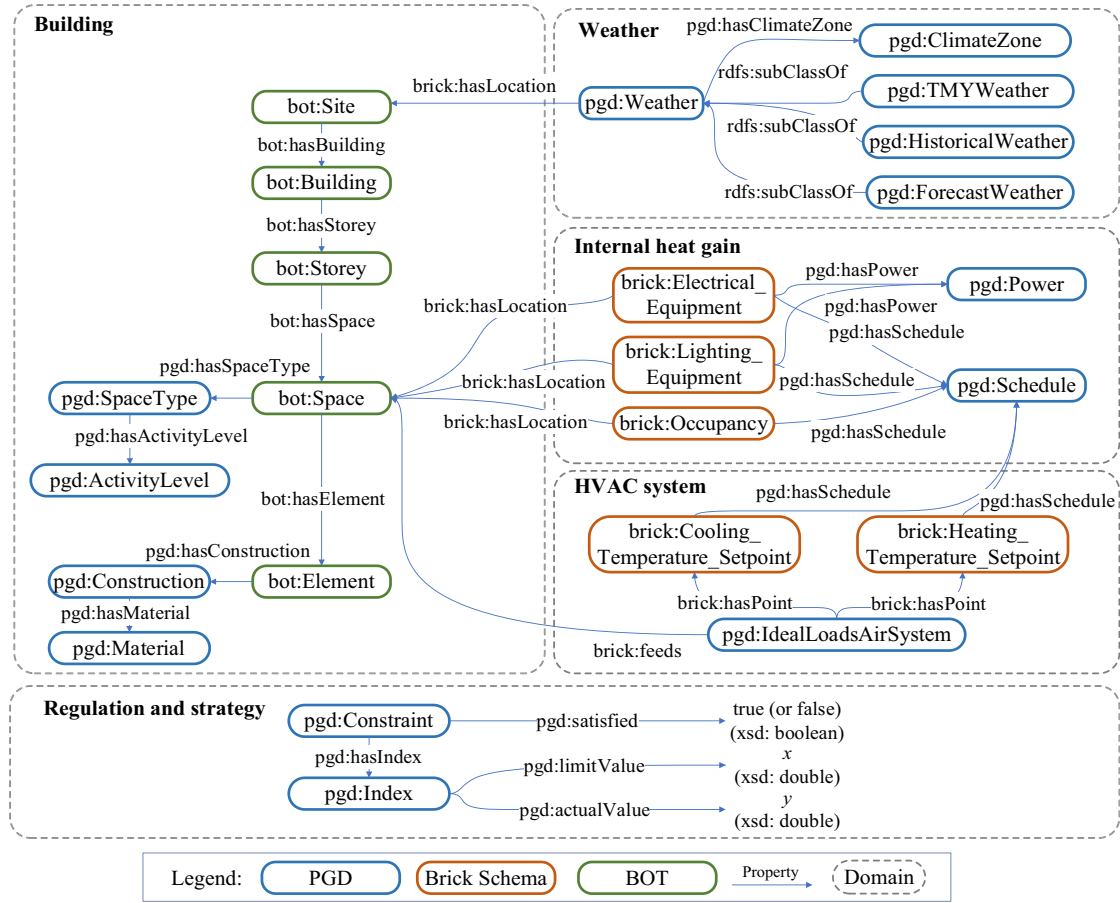


Fig. 3. KG model schema for PGD (PGD-KG schema).

3.2. PGD-KG schema

Previously, we proposed an ontology model for building energy modeling [36] which integrates four domains (weather, building, internal heat gain and HVAC system) by using existing ontology models (Brick Schema [34] and BOT [33]) with necessary extensions. The ontology model [36] supports automated thermal zoning and building energy modeling. To enhance the generalizability of the proposed KG model and to reduce the need for repetitive model development, our KG model schema for PGD leverages the ontology model from [36], with necessary extensions implemented to comprehensively capture and represent the topological relations and functionalities in PGD. Fig. 3 shows the overview of the PGD-KG schema, which covers five key domains in PGD: building, weather, internal heat gain, HVAC system, and regulation and strategy. The components beyond the existing ontology models (Brick Schema and BOT) are assigned a prefix *pgd*. The PGD-KG schema represented in Turtle syntax [39] can be found in Appendix D.

Fig. 4 depicts the components of the PGD-KG schema within the building domain. BOT [33] is adopted, with extensions to describe the building topology. The class *bot:Site* (Fig. 4 (a)) is assigned the property *pgd:hasArea_m2* to support the calculation of site-area-related indices, e.g., plot ratio. The class *bot:Building* (Fig. 4 (a)) stores the information on building types and building directions, and *bot:Space* (Fig. 4 (b)) describes spatial zones, and stores the geometric information (ceiling heights, floor areas, volumes, etc.) of the zones. The topology property *pgd:hasAirFlow* is assigned to the class *bot:Space* to describe the topology relation of airflow among spaces. Each *bot:Space* is connected with the class *pgd:SpaceType* that indicates the space's function and the activity level of occupants in the space.

The class *bot:Element* describes building surfaces (walls, floors, roofs, ceilings and air walls), fenestration surfaces (windows and doors), and shading surfaces. Topology relations in a building surface (Fig. 4 (c)) contain information on surface types, geometries (surface areas, vertex coordinates and normal vectors), boundary conditions (whether the surface is exposed to the outdoor environment), construction and materials (thermal properties, optical properties, etc.). The topology relations in a fenestration surface (Fig. 4 (d)) are similar to those in a building surface but the material properties are different. Air walls are introduced to describe large openings between two spaces, as commonly adopted in modeling software, e.g., EnergyPlus [40]. In the PGD-KG schema, air walls (Fig. 4 (e)) are regarded as a kind of building surface with the construction *AirBoundary*, which is aligned with the setting in EnergyPlus [40]. In a shading surface, the topology relations (Fig. 4 (f)) are also similar to those in a building surface, but the properties only contain information on the vertex coordinates, construction and materials.

The components of the PGD-KG schema in the domains of weather, internal heat gain and HVAC system are shown in Appendix A. *Minor* revisions are made compared with the previous ontology model [36], and details of these components can be found in the Section 3.1 of [36]. The KG components within the regulation and strategy domain will be illustrated in Section 3.4.

3.3. PGD-KG model generation

In current PGD practices, designers create and manipulate parametric design models with parametric design tools (e.g., Grasshopper for Rhino [41] and Dynamo for Revit [42]). A parametric design model

belongs to the class of geometric models that describe building geometry in computational geometry and topology in geometric modeling and graphics [43]. However, for conducting design evaluation tasks such as compliance checking and performance evaluation, additional semantic information, such as building space types and surface materials, which is not encompassed within the geometric model alone is required. A KG provides a solution to the cross-domain integration of geometric and semantic information in PGD. In this section, a PGD-KG model generation method is proposed based on parametric design models with semantic enrichment.

Fig. 5 depicts the workflow of the PGD-KG model generation. Firstly, spaces and surfaces in the parametric design models are translated into

individuals under the classes *bot:Space* and *bot:Element* in the PGD-KG model. Secondly, properties of the spaces and surfaces, e.g., volumes and floor areas of spaces, types, areas, vertex coordinates and normal vectors of surfaces, are identified or calculated. The corresponding KG properties (*pgd:hasVolume_m3*, *pgd:hasFloorArea_m2*, *pgd:hasSurfaceType*, *pgd:hasArea_m2*, *pgd:hasVertex1X_m*, *pgd:hasNormalVectorX_m*, etc.) are also generated. Thirdly, semantic information, e.g., weather, space type, is translated into individuals under the classes *pgd:Weather*, *pgd:SpaceType*, etc. of the PGD-KG model. The semantic information is derived from default templates, which could be reused in various projects, or it is manually input by designers for specific cases. Finally, knowledge inference is conducted based on the generated PGD-KG model, and

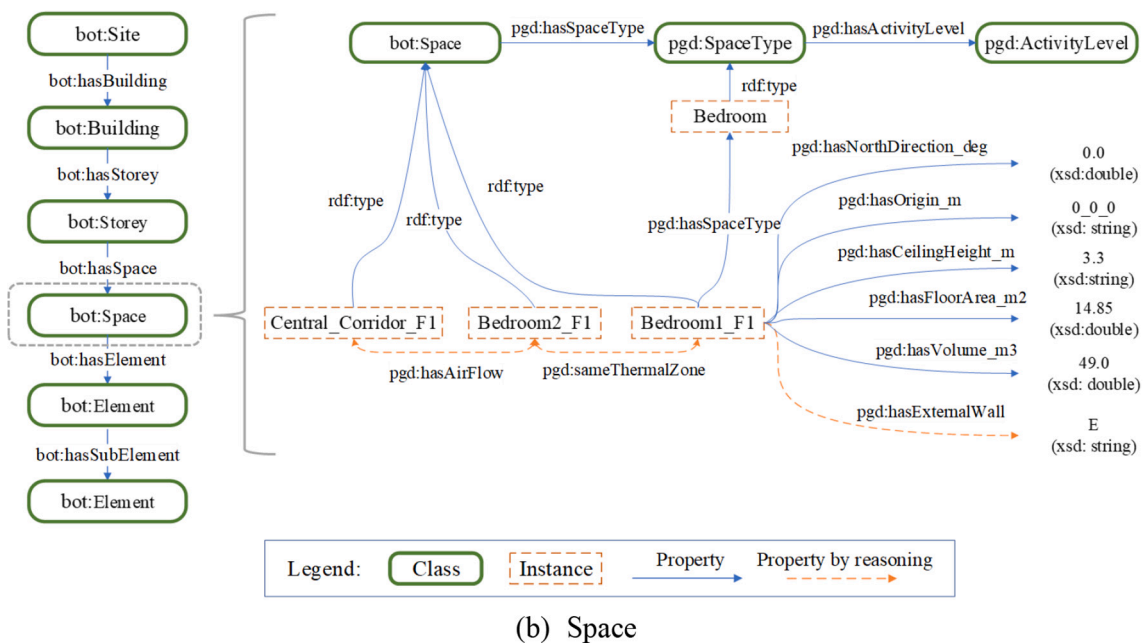
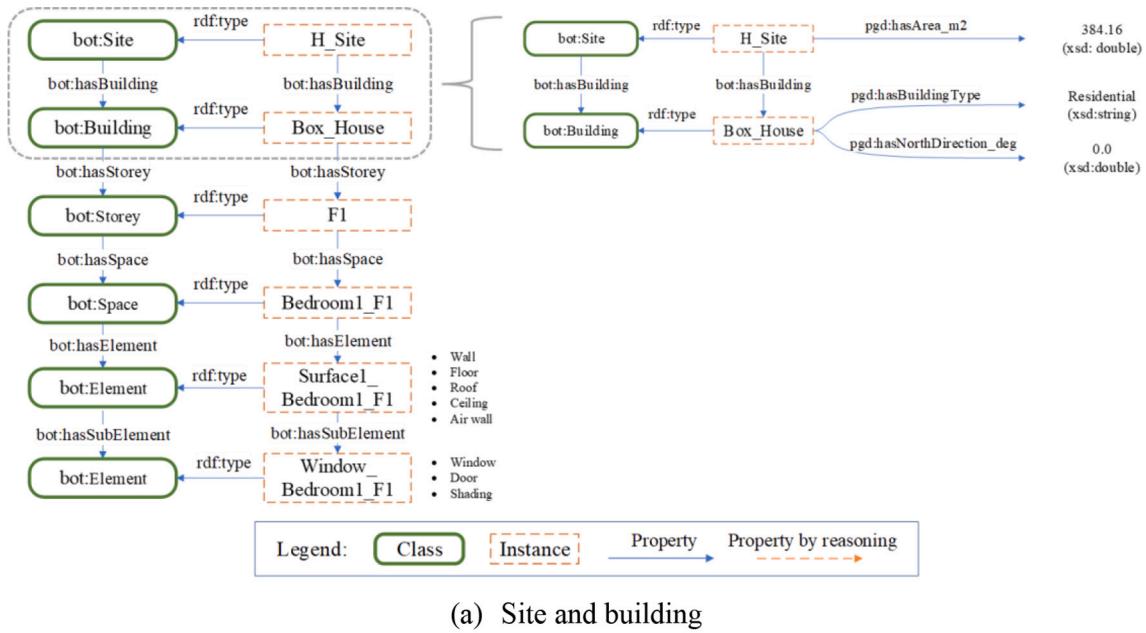


Fig. 4. PGD-KG components within the building domain.

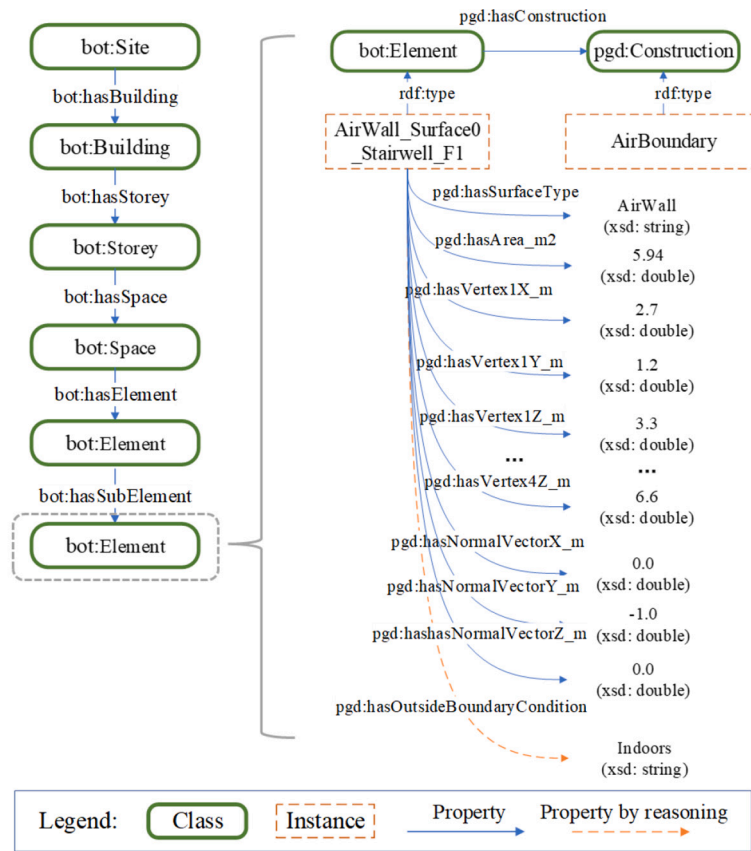


(c) Building surface

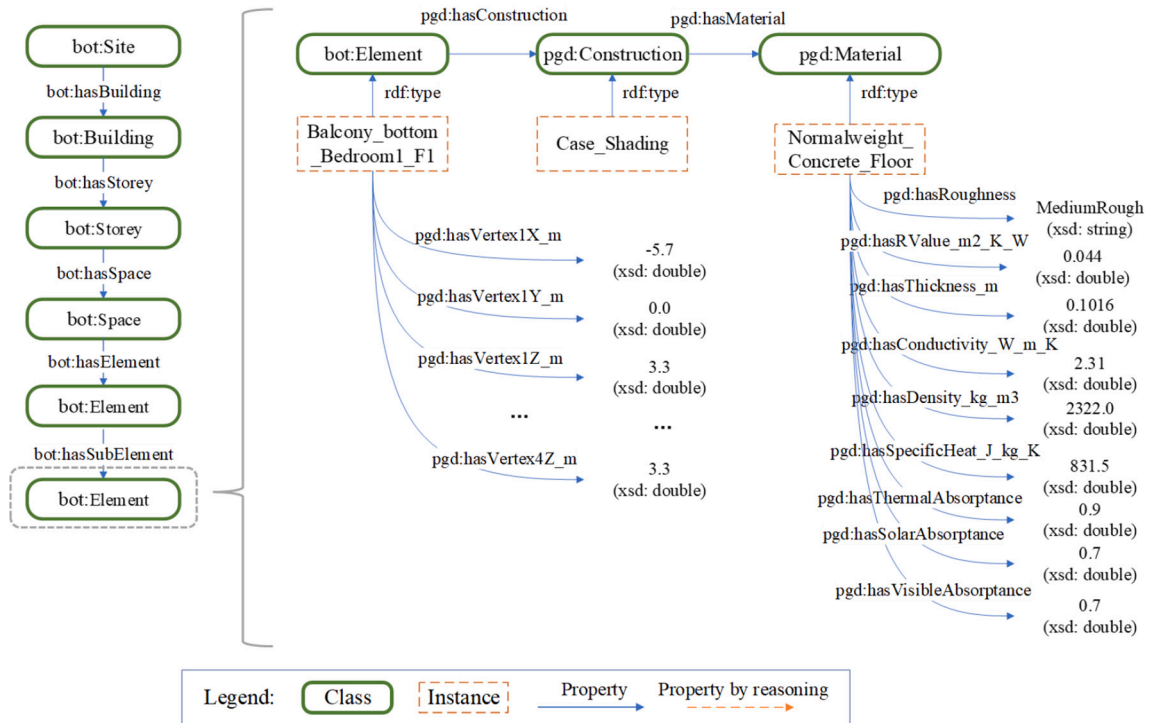


(d) Fenestration surface

Fig. 4. (continued).



(e) Air wall



(f) Shading surface

Fig. 4. (continued).

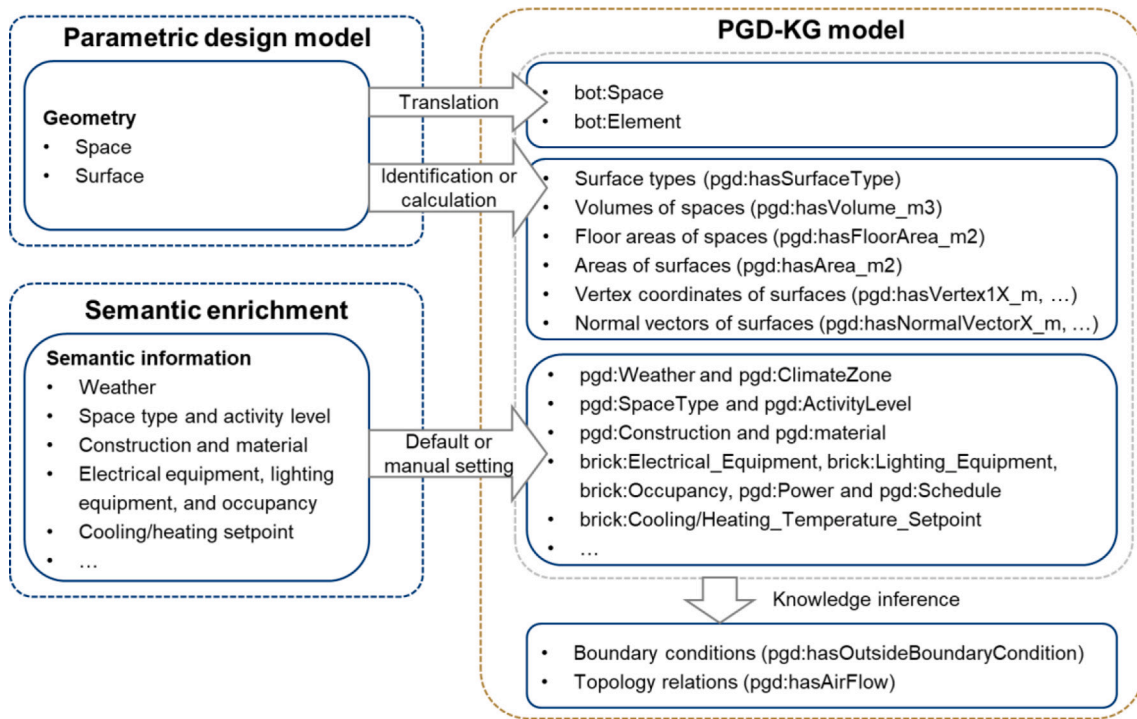


Fig. 5. Workflow of PGD-KG model generation from parametric design models and semantic enrichment.

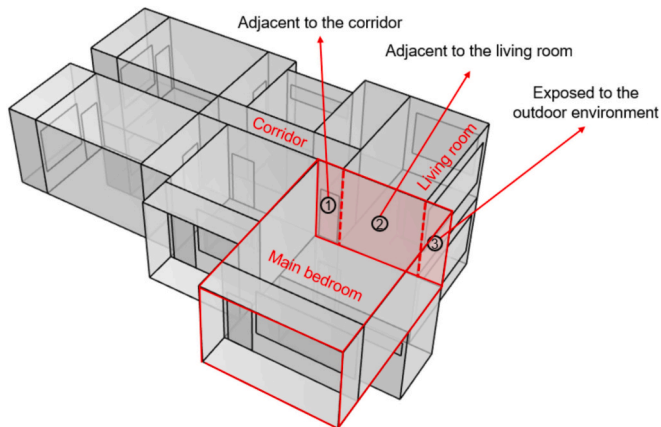


Fig. 6. Example of dividing a building surface.

boundary conditions and topology relations are supplemented to the PGD-KG model.

Algorithm 1 shows the PGD-KG model generation from parametric design models with enriched semantic information. Spaces and building surfaces in the parametric design models are translated and interpreted in Lines 2–33, Algorithm 1. Each space in the parametric design models is identified as one individual under the class *bot:Space* (Line 3, Algorithm 1). Semantic information, e.g., occupancy, equipment, lighting, and HVAC, is added as KG properties of the space individuals (Lines 4–8, Algorithm 1). All the surfaces of the space are extracted (Line 9, Algorithm 1) and are divided according to their adjacent objects (Lines 10–11, Algorithm 1). Fig. 6 depicts one example of dividing a building surface, in which a surface behind the main bedroom (highlighted in red) is divided into three surfaces that are adjacent to the corridor, adjacent to the living room and exposed to the outdoor environment respectively. Existing tools (e.g., Intersect Solids of Ladybug Tools for Grasshopper [44]) can be applied to divide the surface (Line 11, Algorithm 1). Each divided surface is identified as one individual component

under the class *bot:Element* (Line 14, Algorithm 1). The surface types can be identified automatically according to predetermined rules (Lines 16–17, Algorithm 1). For example, a building surface will be identified as a ceiling or roof if the angle between the z-axis and the surface's normal vector is no more than a limited degree (e.g., 30°). Semantic information on construction is added as KG properties of the building surface individuals (Line 18, Algorithm 1). Geometric properties of building surfaces, e.g., surface areas, vertex coordinates and normal vectors, are calculated and added as KG properties of the building surface individuals (Lines 15, 20–24, Algorithm 1). Geometric properties of spaces, e.g., floor areas and space volumes, are calculated and added as KG properties of the space individuals (Lines 26–32, Algorithm 1). Fenestration surfaces in the parametric design models are translated and interpreted in Lines 34–46, Algorithm 1. Each fenestration surface in the parametric design models is identified as one individual under the class *bot:Element* (Line 35, Algorithm 1). Semantic information on surface types and construction is added as KG properties of the fenestration surface individuals (Lines 36–37, Algorithm 1). Geometric properties of the fenestration surface, e.g., surface areas, vertex coordinates and normal vectors, are calculated and added as KG properties of the fenestration surface individuals (Lines 38–43, Algorithm 1). If all the vertices of a fenestration surface are located on a building surface, the fenestration surface is regarded as attached to the building surface, and the “attaching” relations are extracted and the property *pgd:hasSubElement* is added between the building surface and the fenestration surface (Lines 44–45, Algorithm 1). Shading surfaces in the parametric design models are translated and interpreted in Lines 47–52, Algorithm 1. Each shading surface in the parametric design models is identified as one individual under the class *bot:Element* (Line 48, Algorithm 1). The geometric property of the shading surface, namely, the surface vertex coordinates, is calculated and added as a KG property of the shading surface individuals (Lines 49–50, Algorithm 1), and semantic information on construction is added (Line 51, Algorithm 1).

Algorithm 1. PGD-KG model generation from parametric design models with enriched semantics.

Input: Parametric design model M_P , schema-only KG model M_K , semantic information including *Space_Type*, *Construction*, *Occupancy*, *Equipment*, *Lighting*, *HVAC*, etc.

Output: instance-enriched KG model M_K

```

1  wall_ls[ ](and floor_ls[ ], ceiling_ls[ ], etc.) ← [ ]
2  for each Space in  $M_P$  do
3      Space ∈ bot:Space
4      add (Space, pgd:SpaceType, Space_Type) to  $M_K$ 
5      add (Occupancy, brick:hasLocation, Space) to  $M_K$ 
6      add (Equipment, brick:hasLocation, Space) to  $M_K$ 
7      add (Lighting, brick:hasLocation, Space) to  $M_K$ 
8      add (HVAC, brick:feeds, Space) to  $M_K$ 
9      Space_surface_ls[ ] ← explode Space into Space_surfaces
10     for each Space_surface in Space_surface_ls[ ] do
11         building_surface_ls[ ].append(divideSurface(Space_surface))
12     end for
13     for each building_surface in building_surface_ls[ ] do
14         building_surface ∈ bot:Element
15         normal_vector ← calculateNormalVector(building_surface)
16         surface_type ← identifySurfaceType(building_surface)
17         add (building_surface, pgd:SurfaceType, surface_type) to  $M_K$ 
18         add (building_surface, pgd:hasConstruction, Construction) to  $M_K$ 
19         wall_ls[ ](or floor_ls[ ], ceiling_ls[ ], etc.).append(building_surface)
20         surface_area ← calculateArea(building_surface)
21         add (building_surface, pgd:hasArea_m2, surface_area) to  $M_K$ 
22         vertex_coordinate ← getVertex(building_surface)
23         add (building_surface, pgd:hasXYZVertex{i}_m, vertex_coordinate) to  $M_K$ ,  $i = 1, 2, \dots$ 
24         add (building_surface, pgd:hasNormalVector{i}_m, normal_vector) to  $M_K$ ,  $i = X, Y, Z$ 
25     end for
26     floor_area ← 0
27     for floor_surface in floor_ls[ ] do
28         floor_area + ← calculateArea(floor_surface)
29     end for
30     add (Space, pgd: hasFloorArea_m2, floor_area) to  $M_K$ 
31     space_volume ← calculateVolume(Space)
32     add (Space, pgd:hasVolume_m3, space_area) to  $M_K$ 
33 end for
34 for each fenestration_surface in  $M_P$  do
35     fenestration_surface ∈ bot:Element
36     add (fenestration_surface, pgd:hasSurfaceType, Surface_Type) to  $M_K$ 
37     add (fenestration_surface, pgd:hasConstruction, Construction) to  $M_K$ 
38     surface_area ← calculateArea(fenestration_surface)
39     add (fenestration_surface, pgd:hasArea_m2, surface_area) to  $M_K$ 
40     vertex_coordinate ← getVertex(fenestration_surface)
41     add (fenestration_surface, pgd:hasXYZVertex{i}_m, vertex_coordinate) to  $M_K$ ,  $i = 1, 2, \dots$ 
42     normal_vector ← calculateNormalVector(fenestration_surface)
43     add (fenestration_surface, pgd:hasNormalVector{i}_m, normal_vector) to  $M_K$ ,  $i = X, Y, Z$ 
44     building_surface ← findAttachingSurface(fenestration_surface)
45     add (building_surface, pgd:hasSubElement, fenestration_surface) to  $M_K$ 
46 end for
47 for each shading_surface in  $M_P$  do
48     shading_surface ∈ bot:Element
49     vertex_coordinate ← getVertex(shading_surface)
50     add (shading_surface, pgd:hasXYZVertex{i}_m, vertex_coordinate) to  $M_K$ ,  $i = 1, 2, \dots$ 
51     add (shading_surface, pgd:hasConstruction, Construction) to  $M_K$ 
52 end for

```

Define each space as one individual of *bot:Space*.

Add semantic information, including occupancy, equipment, lighting, and HVAC.

Extract space surfaces and divide them according to their adjacent objects.

Define each building surface as one individual of *bot:Element*.

Calculate normal vectors of building surfaces.

Identify surface types.

Add semantic information on construction.

Calculate and add the building surface property of surface area.

Calculate and add the building surface property of vertex coordinate and normal vector.

Calculate and add the space property of floor area and volume.

Identify each fenestration surface in the parametric design models is as one individual of *bot:Element*.

Add semantic information on surface type and construction.

Calculate and add the fenestration surface property of surface areas, vertex coordinates and normal vectors.

Find the building surfaces that the fenestration surfaces are attached to.

Define each shading surface as one individual of *bot:Element*.

Calculate and add the shading surface property of surface vertex coordinates.

Add semantic information on construction.

To fully support the KG-based reasoning for compliance checking and performance evaluation, other knowledge, such as boundary conditions and airflow topology relations, is needed. This implicit knowledge is inferred from the generated PGD-KG model with support of the SPARQL Protocol and RDF Query Language (SPARQL) [45] and Semantic Web Rule Language (SWRL) [46], the query and reasoning languages for KGs. Algorithm 2 shows the inference algorithm for boundary conditions (Lines 2–17, Algorithm 2) and airflow topology relations (Lines 18–20, Algorithm 2). In this study, outdoor surfaces are defined as building or fenestration surfaces that are exposed to outside environmental conditions (e.g., sun exposure and wind exposure), which is aligned with the definition in energy simulation tools e.g., EnergyPlus [40]. For building surfaces, surfaces attached to each other are identified first (Lines 2–5, Algorithm 2). A building surface to which another surface is attached is identified as an indoor surface (Lines 4–6, Algorithm 2), and otherwise, an outdoor surface (Lines 7–8, Algorithm 2). For fenestration surfaces, the number of walls to which the surfaces are attached is calculated first (Line 12, Algorithm 2) by implementing the SPARQL-based reasoning algorithm given in Algorithm 3 in Appendix B. If the fenestration surface has only one attached building surface, it will be regarded as an outdoor surface (Lines 13–14, Algorithm 2), and otherwise, as an indoor surface (Lines 15–16, Algorithm 2). Airflow topology relations are obtained (Lines 19–20, Algorithm 2) by the SWRL-based reasoning given in Algorithm 4 in Appendix B, and three kinds (airflow between a space and a fenestration surface, airflow between a space and an air wall, and airflow between an air wall and its adjacent air wall) can be extracted. Fig. 7 depicts an example of airflow topology relations in a part of a building after the inference.

Algorithm 2. Knowledge inference of boundary conditions and airflow topology relations.

Input: Instance-enriched KG model M_K

Output: Instance-enriched KG model with inferred knowledge M_{K-i}

```

1  // Knowledge inference of boundary conditions
2  for each building_surface1 ∈ bot:Element do
3      for each building_surface2 ∈ bot:Element do
4          if building_surface1 ≠ building_surface2 and building_surface1 is attached to building_surface2 then
5              add (building_surface1, bot:adjacentElement, building_surface2) to  $M_{K-i}$ 
6              add (building_surface1, bot:hasOutsideBoundaryCondition, "Indoors") to  $M_{K-i}$ 
7          else
8              add (building_surface1, bot:hasOutsideBoundaryCondition, "Outdoors") to  $M_{K-i}$ 
9      end for
10 end for
11 for each fenestration_surface ∈ bot:Element do
12     num ← calculateNumberOfAttachedWall(fenestration_surface)
13     if num = 1 then
14         add (fenestration_surface, bot:hasOutsideBoundaryCondition, "Outdoors") to  $M_{K-i}$ 
15     else
16         add (fenestration_surface, bot:hasOutsideBoundaryCondition, "Indoors") to  $M_{K-i}$ 
17     end for
18 // Knowledge inference of airflow topology relations
19 inference_triples ← topologyReasoning( $M_K$ )
20 add inference_triples to  $M_{K-i}$ 

```

Identify building surfaces attached to each other.

Determine outside boundary conditions of building surfaces.

Calculate the number of attached wall(s) of each fenestration surface using Algorithm 3.

Determine outside boundary conditions of fenestration surfaces.

Obtain airflow topology relationships using Algorithm 4.

3.4. Reasoning algorithms for compliance checking

In early design stages, compliance checking plays a crucial role in verifying the adherence of building designs to regulatory requirements and compliance with pertinent codes and standards [47]. To integrate compliance checking into PGD, we propose in this section a KG

component and a set of KG-based cross-domain reasoning algorithms for compliance checking. Fig. 8 shows the KG component for compliance checking in the PGD-KG schema. The class *pgd:Constraint* describes the constraints specified by the building codes and standards, and the class *pgd:Index* indicates the index that should be calculated and inspected for code compliance. An example of a height limit is shown in Fig. 8, wherein the upper limit of the building height is preset and the actual height is calculated by the reasoning of the PGD-KG model. The satisfaction of the height limit constraint can be determined by comparing the actual height with the prescribed height limit through reasoning, and the property *pgd:satisfied* is then assigned to indicate whether the constraint has been met or not. The specific algorithms used for reasoning will be further elaborated in the subsequent paragraphs.

To illustrate how the reasoning algorithms achieve automated compliance checking, four regulatory requirements, height limit, percentage of site coverage, plot ratio and site boundary, are taken as an example in this section, and these requirements are applied to the illustrative example in Section 4. Table 3 summarizes the limit values of the height limit, percentage of site coverage and plot ratio set out in the Building (Planning) Regulations of Hong Kong [48] and the project-specific requirement of the site boundary used in the illustrative example.

To conduct compliance checking in the knowledge-informed optimization framework, the first step is to retrieve relevant data from the PGD-KG model and calculate the actual values of the constraint indices using the reasoning rules. Algorithms 5–8 (listed in Appendix B) show the reasoning algorithms in SPARQL for the actual height, percentage of site coverage, plot ratio and site boundary respectively. The reasoning algorithm for actual height (Algorithm 5) obtains the maximum of the z coordinates of the surface vertices as the height of the building. In the reasoning algorithm for actual percentage of site coverage (Algorithm

6), the site area is retrieved (Lines 3 and 4, Algorithm 6) and the floor area of the ground floor is calculated by the sum of the query results of the floor areas of all the spaces on the ground floor (Lines 6–12, Algorithm 6). Finally, the actual percentage of site coverage is calculated (Line 14, Algorithm 6). The reasoning algorithm for the actual plot ratio (Algorithm 7) is similar to that for the actual percentage of site coverage.

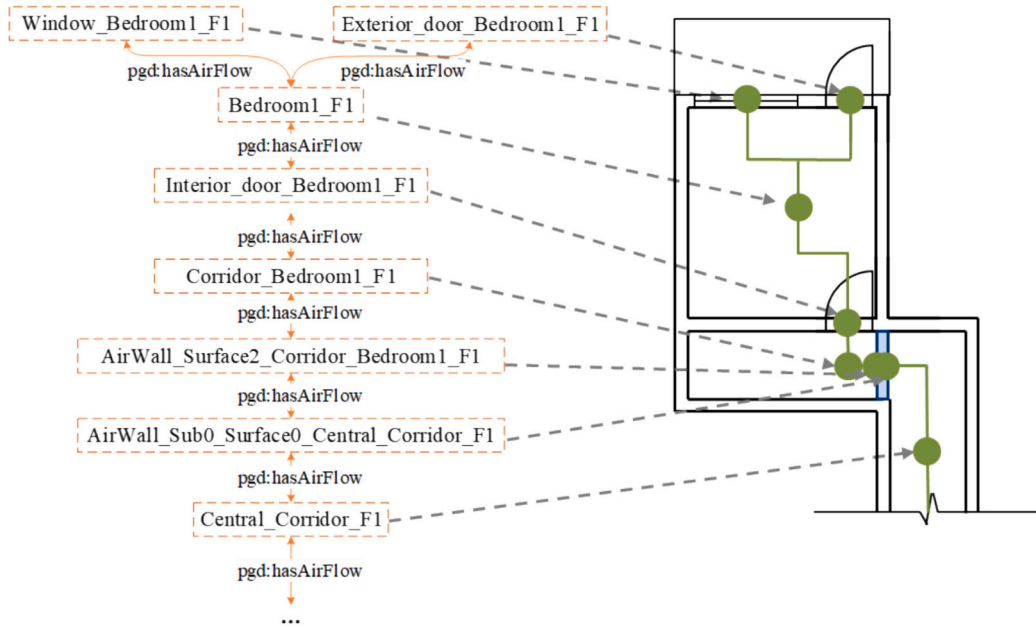


Fig. 7. Example of airflow topology relations.

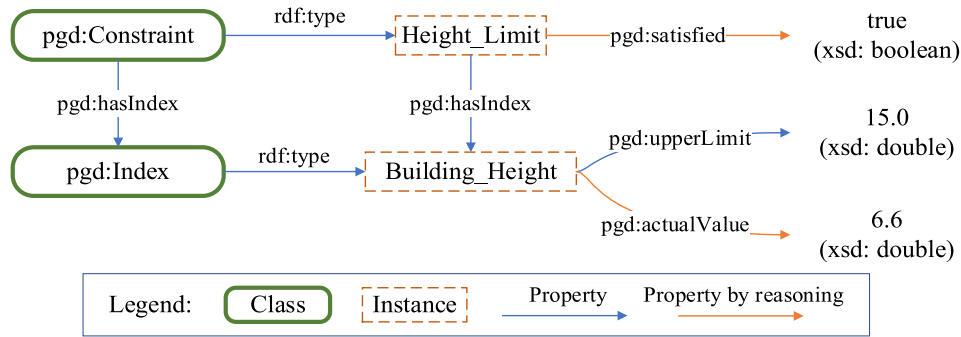


Fig. 8. KG component for compliance checking.

Table 3
Regulatory requirements of the illustrative example.

Index	Height limit	Percentage site coverage	Plot ratio	Site boundary
Calculation	max (Z_coordinate)	ground floor area / site area	total floor area / site area	max(X_coordinate) – min(X_coordinate) and max(Y_coordinate) – min (Y_coordinate)
Limit	≤ 15 m	≤ 0.666	≤ 3.3	≤ 19.6 m × 19.6 m

The site area is retrieved (Lines 3 and 4, Algorithm 7), the total floor area is calculated by the sum of the query results of the floor areas of all the spaces (Lines 6–11, Algorithm 7), and finally, the actual plot ratio is calculated (Line 13, Algorithm 7). The reasoning algorithm for actual site boundary (Algorithm 8) retrieves the x and y coordinates of the vertices (Lines 10–12, 19–21, 27–29 and 36–38, Algorithm 8), and subsequently conducts coordinate transformation from the building coordinate to the true north coordinate (Fig. 9) (Lines 13–14, 22–23, 30–31 and 39–40, Algorithm 8), finally obtaining the ranges of the x and y coordinates (Lines 45–46, Algorithm 8).

The second step is to assess the compliance of the constraints by reasoning, and Algorithm 9 (listed in Appendix B) shows the reasoning algorithm in SWRL. Because there is no conditional structure in SWRL, the assessment of compliance and noncompliance should be written in two parallel strings (Lines 1–8 and Lines 9–16, Algorithm 9, respectively). The following elaboration takes the compliance assessment for the height limit as an example. The property *pgd:actualValue* of the individual *Building_Height* is added to the PGD-KG model (Fig. 8) after the

implementation of Algorithm 5. The property *pgd:upperLimit* of the individual *Building_Height* is retrieved (Line 5 and Line 13, Algorithm 9) and compared with the property *pgd:actualValue* (Line 7 and Line 15, Algorithm 9). The property *pgd:satisfied* of the individual *Height_Limit* is then inferred by the comparison to indicate height limit compliance (Line 8 and Line 16, Algorithm 9).

3.5. Reasoning algorithms for performance evaluation

Design strategies act as valuable guiding principles for architects designing sustainable buildings. Conventionally, architects rely on these strategies to manually inform the design process. In this section, we present a collection of algorithms that automate the performance evaluation by utilizing cross-domain reasoning of PGD-KG models. Fig. 10 elaborates on the workflow of extracting design strategies from the literature and translating them into machine-readable reasoning algorithms. Firstly, three strategies are derived from three books on sustainable building design, *Sun, Wind, and Light* [10], *101 Rules of Thumb*

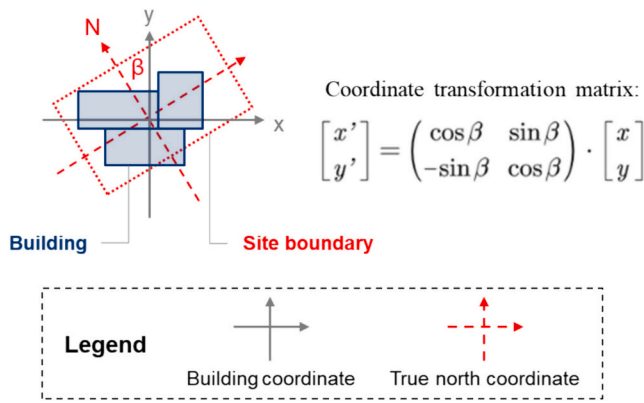


Fig. 9. Coordinate transformation in the reasoning algorithm for site boundary.

for Low Energy Architecture [11] and *The Green Studio Handbook: Environmental Strategies for Schematic Design* [12]. Table 4 shows a summary of the design strategies. Secondly, each strategy is translated into a machine-readable algorithm (Algorithms 10–12, listed in Appendix B) that supports the cross-domain reasoning of the PGD-KG models. Each strategy has an indicator to represent the degree that the strategy can be achieved. The indicators can only be obtained after the form generation and hence they are used to select design cases after generation instead of being taken as constraints in the generation process.

Algorithm 10 shows the reasoning algorithm in SPARQL for the shape factor, which is the indicator of Strategy 1. The volume of the building is calculated (Lines 3–5, Algorithm 10) and the areas of the exterior surfaces of walls, roofs and floors are retrieved from the PGD-KG model and aggregated (Lines 6–28, Algorithm 10). The shape factor is calculated by the ratio of the total exterior surface area and total building volume (Line 29, Algorithm 10). Algorithm 11 shows the reasoning algorithm in SPARQL for the number of nodes in the overall airflow path, the indicator of Strategy 2, which is counted by the number of nodes between two exterior windows. Finally, Algorithm 12 shows the reasoning algorithm in Python for the projection area of windows on the south and north side, which is the indicator of Strategy 3. The areas of windows on the south and north side are retrieved by the implementation of a SPARQL query (Lines 2–9, Algorithm 12) and the projection area is calculated subsequently (Lines 11–21, Algorithm 12).

Following the execution of Algorithms 10–12, values of the three

indicators can be derived, and the design cases are selected according to the indicator values. To determine the number of selected cases, a hyperparameter called the threshold is introduced to the knowledge-informed optimization framework. Design cases whose indicator values satisfy Criterion (1) (if a larger value is desired) or Criterion (2) (if a smaller value is desired) are selected. Subsequently, performance simulations will be conducted to obtain the detailed performance metrics of the selected cases for the optimization process.

$$I \geq I_{\max} - t \cdot (I_{\max} - I_{\min}) \quad (1)$$

$$I \leq I_{\min} + t \cdot (I_{\max} - I_{\min}) \quad (2)$$

in which, I is the value of the indicator, $I_{\max}$ is the maximum value of the indicator among all the design cases, $I_{\min}$ is the minimum value of the indicator among all the design cases, and t is the threshold (%) decided by the users at the beginning of the generation.

4. Illustrative example

4.1. Background of the design project

A hypothetical design project was created to illustrate the implementation of the proposed knowledge-informed optimization framework. A 2-story detached house using prefabricated modular techniques in a pilot design study was to be constructed in a square site with a boundary of 19.6 m × 19.6 m on a campus in Hong Kong (Fig. 11). The design stipulated that the ground floor should contain one living room, one kitchen, one bathroom and one bedroom, while the first floor should contain one living room, one main bedroom, one bathroom, two bedrooms and one study.

Fig. 12 illustrates the modeling rule of the prefabricated modular detached house. The main bedroom, bedrooms, bathrooms, study, kitchen, living rooms and staircase were set as prefabricated modular units whose sizes were fixed. The locations of the staircase and living rooms were settled according to the design constants listed in Table 5, while the positions and orientations of the other prefabricated modular units varied among five slots (A, B, C, D and E). The prefabricated modular units in slots A, C and D had two directions (along the x-axis and y-axis of the building coordinates), while those in slot B and E were only directed along the y-axis. Considering the structural requirements and correspondence of drainage pipes, the main bedroom, bedrooms, bathrooms, study, and kitchen modules were arranged in five

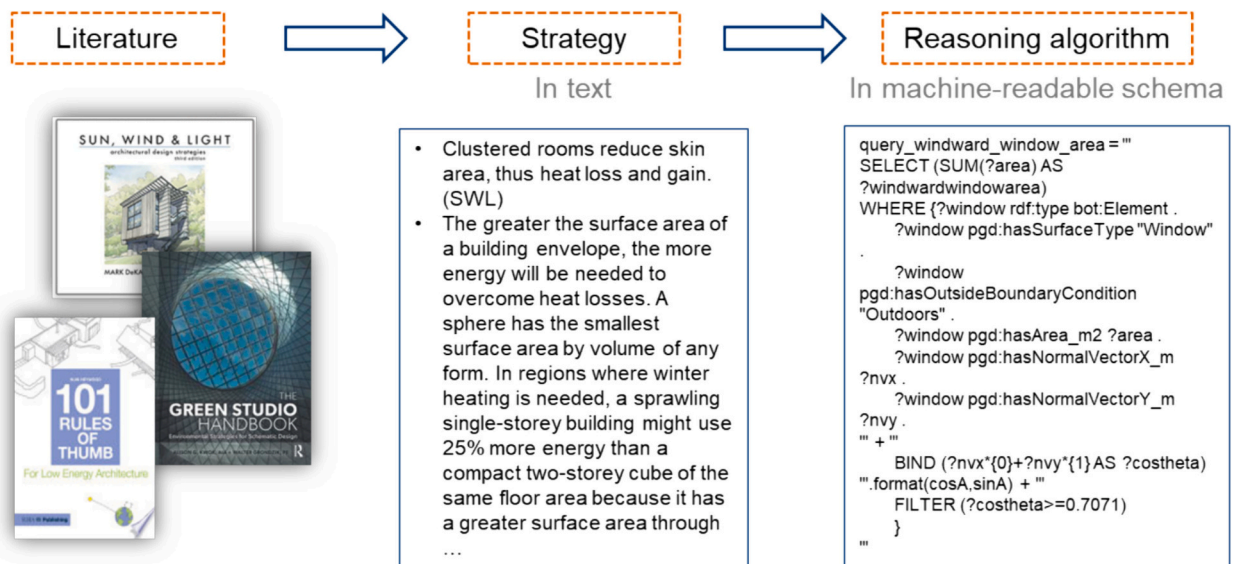


Fig. 10. Extraction of design strategies from the literature and translation into machine-readable reasoning algorithms.

Table 4
Design strategies derived from the literature.

Strategy	Original content	Interpretation	Indicator (goal)	Performance
1	“Clustered rooms reduce skin area, thus heat loss and gain.” (G. Brown & M. DeKay, 2013, p. 142) [10] “The greater the surface area of a building envelope, the more energy will be needed to overcome heat losses. A sphere has the smallest surface area by volume of any form. In regions where winter heating is needed, a sprawling single-story building might use 25% more energy than a compact two-story cube of the same floor area because it has a greater surface area through which heat is lost. A slightly elongated solar-oriented form provides the best balance between heat loss and beneficial solar gain.” (H. Heywood, 2019, p. 76) [11]	The area of exterior envelopes should be as small as possible.	Shape factor (minimize)	Cooling and heating
2	“Permeable buildings can combine open plans and sections for cross-ventilation, stack-ventilation, or both.” (G. Brown & M. DeKay, 2013, p. 145) [10]	The ventilation path should be sufficiently short and the resistance should be sufficiently low.	Number of nodes in overall airflow path (minimize)	Cooling
3	“Daylight apertures can be enlarged to increase interior illumination levels.” (G. Brown & M. DeKay, 2013, p. 251) [10] “Arrange program elements into a footprint that maximizes wall area (surface area to volume ratio)—using, for example, U-shaped buildings, courtyards/atria, long thin building plans. Maximize opportunities for daylighting without direct solar radiation by focusing upon the north and south orientations as prime locations for daylight apertures. East and west apertures require careful consideration of shading devices to reduce the potential for direct glare and unwanted solar gains.” (A.G. Kwok, W. Grondzik, 2018, p. 98) [12]	The light-transmitting windows should be mainly directed to the south or north and be sufficiently large.	Projection area of light-transmitting windows on the south and north side (maximize)	Daylighting and cooling

combinations and allocated to the five slots according to the design variables listed in Table 5. Main corridors were set in the center of the ground floor and the first floor, and auxiliary corridors were set beside the prefabricated modular units if the units were directed along the x-axis. The regulatory requirements of the design case included height limit, percentage of site coverage, plot ratio and site boundary, as listed in Table 3. The goal of the design project was to generate the optimal layouts of the module combinations that would minimize the cooling energy consumption and maximize the daylighting performance of the detached house.

4.2. Framework implementation and experiment design

The proposed knowledge-informed optimization framework was implemented to generate optimal layouts of the module combinations based on the cooling energy consumption and daylighting performance of the detached house. The procedure followed the workflow shown in Fig. 2 and was programmed in Python. Firstly, the parametric design models of the detached house were developed using the Python library rhino3dm [49], which offers geometric manipulation functions that can be used to create and modify parametric Rhino 3D models in Python. Secondly, PGD-KG models were generated based on the parametric design model and enriched semantics using Algorithms 1–4. The Python library rdflib [50] was adopted to construct the PGD-KG models and conduct reasoning for knowledge inference. Thirdly, reasoning for compliance checking of the four regulatory requirements (Table 3) was conducted using Algorithms 5–9, and reasoning for performance evaluation based on the three design strategies (Table 4) was conducted using Algorithms 10–12. The Python library rdflib [50] was also adopted to conduct reasoning for the compliance checking and performance evaluation. Fourthly, simulations in terms of cooling energy consumption and daylighting were conducted using EnergyPlus [51] and Radiance [52] respectively for the cases that passed the compliance check and performance evaluation. Annual cooling load (ACL) and useful daylight illuminance (UDI) [53] were adopted as the performance metrics of the cooling energy consumption and daylighting performance respectively. Finally, optimization was conducted to find the Pareto-optimal solutions that minimize the cooling energy consumption and maximize the daylighting performance of the detached house. We chose Non-Dominated Sorting Genetic Algorithm II (NSGA-II) as the

optimization algorithm in the illustrative example, because NSGA-II is the most used meta-heuristic optimization algorithm in current PGD practice [4].

The simulation and optimization settings are listed in Appendix C. The settings of construction and material (Table C.1. 1), internal heat gain (Table C.1. 2), and schedule (Table C.1. 3) were referenced to mid-rise residential buildings in Climate Zone 2 in the ASHRAE 90.1 Standard [54]. The cooling setpoint was 25.5 °C according to the recommendation promulgated by The Government of the Hong Kong Special Administrative Region [55]. According to a study on design criteria for the built thermal environment for hot summer and warm winter zones of China [56], the acceptable range of indoor air temperature for naturally ventilated urban buildings is 18.0–28.5 °C. Consequently, an airflow network was set in the simulations, and exterior windows would be opened if the outdoor air temperature were between 18.0 and 28.5 °C and the outdoor air temperature were lower than the indoor air temperature. The grid of the daylighting simulations was set as 0.6 m × 0.6 m, and the height of the reference planes was set as 0.76 m from the floors according to the Leadership in Energy and Environmental Design (LEED) guidelines for building design and construction [57]. Illuminance within 100–2000 lx was regarded as useful illuminance [53]. The parameters of the NSGA-II algorithm for optimization (Table C.1. 4) were fine-tuned before the implementation of the proposed framework.

Experiments were designed and conducted to evaluate the performance of the proposed framework. Firstly, we studied the sizes of the solution spaces selected by the reasoning algorithms for performance evaluation under different thresholds (75%, 50%, 40%, 30% 20% and 10%). To analyze the sizes of the selected solution spaces and determine whether the selected cases encompassed all Pareto-optimal solutions, two metrics were employed: the ratio of selected solutions (the number of solutions selected by the reasoning algorithms divided by the total number of cases) and the ratio of selected Pareto-optimal solutions (the number of selected Pareto-optimal solutions divided by the number of Pareto-optimal solutions in the ground truth). Secondly, the accuracy and computational time of the knowledge-informed optimization framework (reasoning-based performance evaluation + NSGA-II optimization) under different thresholds (75%, 50%, 40%, 30% and 20%) were studied by comparing it with the general optimization framework (optimization using NSGA-II without the reasoning-based performance evaluation). Validity (percentage of the number of the Pareto-optimal

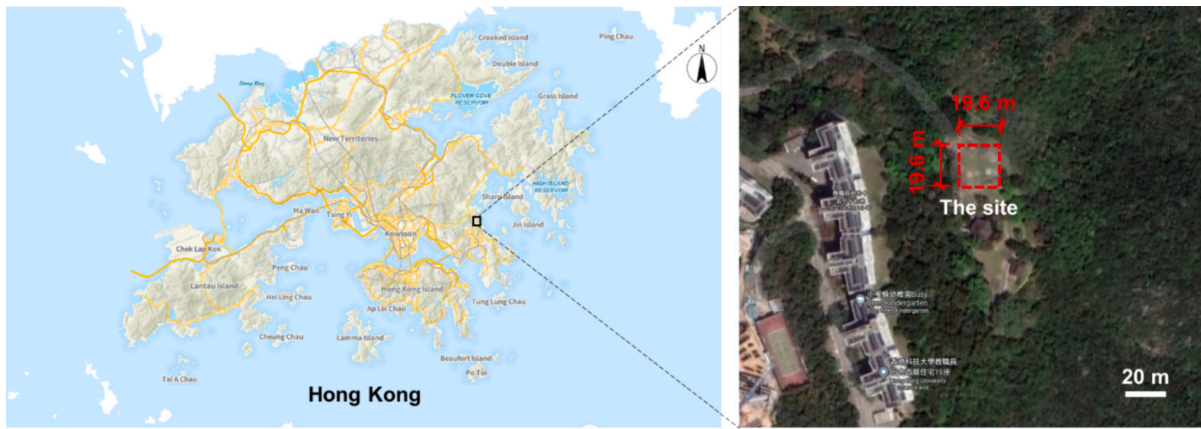


Fig. 11. Site of the design project.

solutions obtained by the algorithms to the number of globally optimal solutions) and computational time were adopted to analyze the accuracy and speed of the proposed framework under different thresholds. The computational time encompassed the time for PGD-KG generation, reasoning for compliance checking and performance evaluation, simulation, and optimization. In addition, all the cases (7680 in total) were simulated with EnergyPlus and Radiance to obtain their ACL and UDI performance metrics and find the ground truth of the Pareto-optimal solutions. The software Protégé [58] was used for the visualization of the PGD-KG models. A desktop computer was used to conduct the experiments and the specifications are as follows: Intel Core CPU i7-6700 at 3.40 GHz and 8 cores, 32 GB RAM, and Windows 64 operating system. The versions of software or tools used are as follows: Python 3.9.13, EnergyPlus 22.2.0, Radiance 5.4a and Protégé 5.5.0.

4.3. Results

4.3.1. PGD-KG models and compliance checking

The PGD-KG models of all the design cases could be generated from their parametric design models with enriched semantics. Fig. 13 shows the classes, properties, individuals, and metrics of the PGD-KG model of Case 6725, which are visualized in Protégé. The PGD-KG model of this example (Case 6725) represented in Turtle syntax [39] can be found in Appendix D.

The results of compliance checking showed that all the cases satisfied the regulatory requirements for the percentage of site coverage, plot ratio and height limit, while 64 cases did not satisfy the requirement for the site boundary. Fig. 14 shows one case (Case 1068) that did not pass the compliance check for the site boundary, with the reasoning of the compliance check visualized in Protégé. In total, 7616 cases passed the compliance checking and entered the remaining stages of the knowledge-informed optimization framework. The results of the 7616 cases are elaborated on and discussed in the following sections.

4.3.2. Solution space

Fig. 15 shows the whole solution space, which contained the 7616 cases and the Pareto-optimal solutions of the ground truth obtained by the simulations of all the cases. Due to the discrete nature of the design variables, the solution space was likewise discrete. There were 8 module combinations (i.e., the Pareto-optimal solutions) that minimized the cooling energy consumption and meanwhile maximized the daylighting performance of the detached house. The drawings, design variables, performance metrics and reasoning indicators of the Pareto-optimal solutions are shown in Fig. 15.

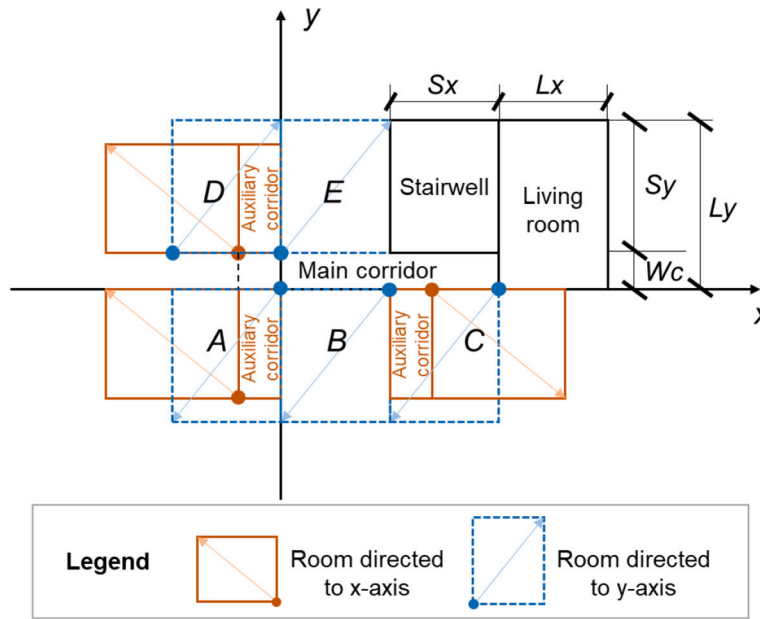
Table 6 shows the ratio of selected solutions and the ratio of selected Pareto-optimal solutions, and Fig. 16 shows the cases selected by the reasoning algorithms for performance evaluation under the thresholds

of 75%, 50%, 40%, 30%, 20% and 10% (markers in orange), by comparison with all the design cases in the whole solution space (markers in blue). As the threshold became more stringent (i.e., smaller in magnitude), the ratio of the selected solutions became smaller and the selected solution space contracted, orienting towards lower ACL and higher UDI values. When the threshold was equal to or above 40%, all the Pareto-optimal solutions were included in the selected solution space. When the threshold was 30% or 20%, one Pareto-optimal solution (Case 5) was not included. Case 5 (Fig. 15) satisfied Strategy 1 and Strategy 2, but failed to comply with Strategy 3 if the threshold was more stringent than 40%. When the threshold was 10%, no solution was selected, showing that no case could satisfy all the strategies with such a stringent threshold. The above results demonstrate that to obtain a smaller solution space, a more stringent threshold should be adopted. Meanwhile, more stringent thresholds will lead to some missing Pareto-optimal solutions, and extremely stringent thresholds will lead to no solution being selected.

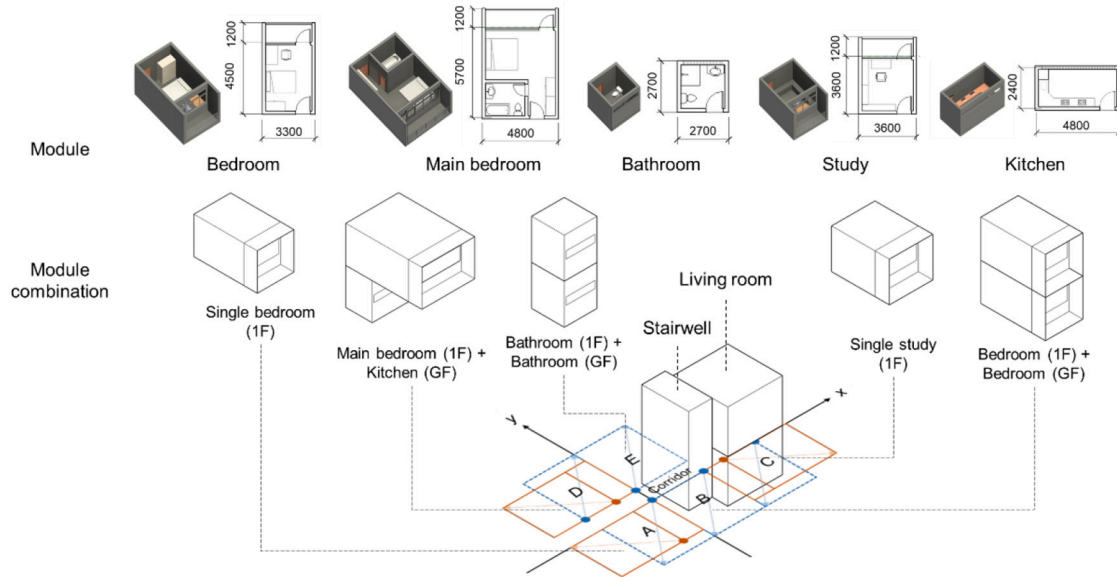
To summarize, the strategy-driven reasoning algorithms for performance evaluation could narrow the solution space of the optimization while including an adequate number of Pareto-optimal solutions ($\geq 87.5\%$) by adopting proper thresholds.

4.3.3. Accuracy and speed

Table 7 shows the accuracy and speed of the knowledge-informed optimization framework (reasoning-based performance evaluation + NSGA-II optimization) under the thresholds of 75%, 50%, 40%, 30% and 20% compared with the general optimization framework (optimization using NSGA-II without the reasoning-based performance evaluation). The threshold of 10% was not included in the analysis since no solution was earlier selected under this threshold. The results of the validity under different thresholds were the same as those of the ratio of selected Pareto-optimal solutions, showing that NSGA-II was able to find all the Pareto-optimal solutions in the selected solution space. Compared with the general optimization framework, the knowledge-informed optimization framework reduced the computational time substantially. The general optimization framework took 135,512 s (approximately 37.6 h) to finish the optimization process, while the proposed framework could reduce the computational time by 73.25% (to approximately 10.1 h) with the threshold of 20%. As the threshold was more stringent (i.e., smaller in magnitude), less computational time was consumed. As illustrated in Section 4.3.2, more stringent thresholds mean smaller solution spaces and hence less simulation and optimization time in the overall framework. However, there was a minor difference in the reduction of computational time when the threshold was below 30%, indicating that when the solution space reached a size equivalent to a certain value, further reduction in the solution space would not enhance the efficiency of NSGA-II. The plausible reason for no improvement of



(a) Slots of the prefabricated modular units.



(b) Modules and module combination of the detached house (The dashed lines show one example arrangement).

Fig. 12. The modeling rule of the prefabricated modular detached house.

NSGA-II performance was that when the solution space becomes smaller, the optimization process got closer to the stage of local search. As mentioned in [59], NSGA-II often generates similar individuals, and hence leads to a decrease in local search capability. The repeated individual generation was also observed in our experiment. Consequently, although more stringent threshold can narrow the solution space significantly, less improvement will be achieved due to the defect of NSGA-II in local search capability.

Overall, the proposed knowledge-informed optimization framework could obtain an adequate number of Pareto-optimal solutions ($\geq 87.5\%$)

while reducing computational time by approximately 73%, compared with the general optimization framework.

4.4. Discussion

4.4.1. Analysis of the proposed framework

The results of the illustrative example in Section 4.3 indicate that the proposed knowledge-informed optimization framework is able to obtain an adequate number of Pareto-optimal solutions ($\geq 87.5\%$) while reducing computational time (by approximately 73%), compared with

Table 5

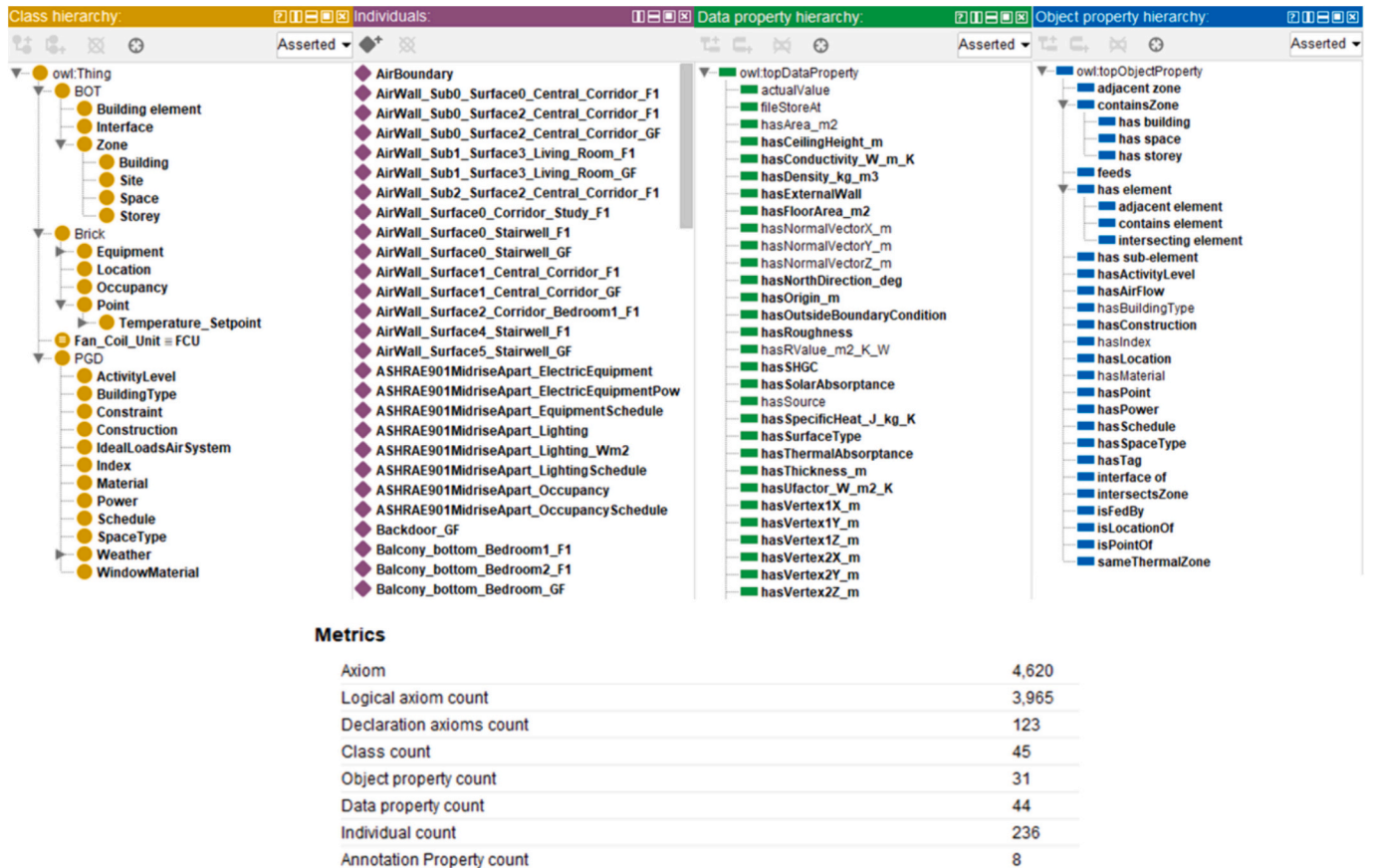
Design parameters of the prefabricated modular detached house.

Parameter	Description	Value
Constant	W_c Corridor width	1.2 m
	H Floor height	3.3 m
	L_x Living room width	3.0 m
	L_y Living room depth	4.8 m
	S_x Stairwell width	1.8 m
	S_y Stairwell depth	3.6 m
Decision variable	S_a Combination in slot A	Range: [0, 4], step: 1
	S_b Combination in slot B	
	S_c Combination in slot C	0: Single bedroom (1F);
	S_d Combination in slot D	1: Main bedroom (1F) + Kitchen (GF);
	S_e Combination in slot E	2: Bathroom (1F) + Bathroom (GF);
		3: Single study (1F);
		4: Bedroom (1F) + Bedroom (GF).
	D_a Direction of the combination in slot A	Range: [0,1], step: 1
	D_c Direction of the combination in slot C	0: along y-axis; 1: along x-axis.
	D_d Direction of the combination in slot D	
	O Orientation	Range: [0,360°], step: 45°

the current optimization framework. The proposed framework can save time because the reasoning algorithms can effectively narrow down the solution space of the optimization while ensuring the inclusion of an adequate number of Pareto-optimal solutions by adopting proper thresholds. The narrower solution space leads to fewer cases to be simulated and analyzed, and hence the time taken by simulations and the optimization is reduced. The process of narrowing the solution space

aligns with the desired direction to minimize ACL and maximize UDI (as observed in the illustrative example, involving a shift towards a lower ACL and higher UDI). The reasoning algorithms for performance evaluation are derived from expert knowledge that guides sustainable building design, thereby offering a clear explanation for the narrowing process. The rationale of the knowledge-guided space narrowing is similar to Machairas et al.'s idea (2014) [8] that “the best tools and algorithms for the building design problem are the ones that enable researchers to use their expert knowledge in order to minimize the size of the solution space or drive the search to the right direction” (p. 105). We have realized their idea in this study by integrating expert knowledge into the PGD optimization framework by developing a KG-based model for PGD and implementing KG-based reasoning for case selection based on compliance checking and performance evaluation.

Threshold is the only hyperparameter that should be decided by the user in the proposed knowledge-informed optimization framework. More stringent thresholds lead to narrower solution spaces, fewer simulation cases and less computational time, but may result in missing some Pareto-optimal solutions. The plausible rationale for the missing Pareto-optimal solutions lies in the strategies that support the reasoning algorithm. As shown in Fig. 16, although the solution space was narrowed towards a lower ACL and higher UDI, not all the cases with a lower ACL and higher UDI were included. The excluded cases did not satisfy all the strategies but had better energy and daylighting performance. The literature-derived strategies belong to rules of thumb, heavily rely on human experience, and may contain bias and errors. As the threshold becomes more stringent, the bias and errors may be magnified. According to Jin's review [60], in general, fitness evaluations characterized by high fidelity tend to be more time-intensive while fitness evaluations with low fidelity are typically less time-consuming, and hence users should strike a balance between accuracy and

**Fig. 13.** Classes, properties, individuals, and metrics of the PGD-KG model of Case 6725, visualized in Protégé.

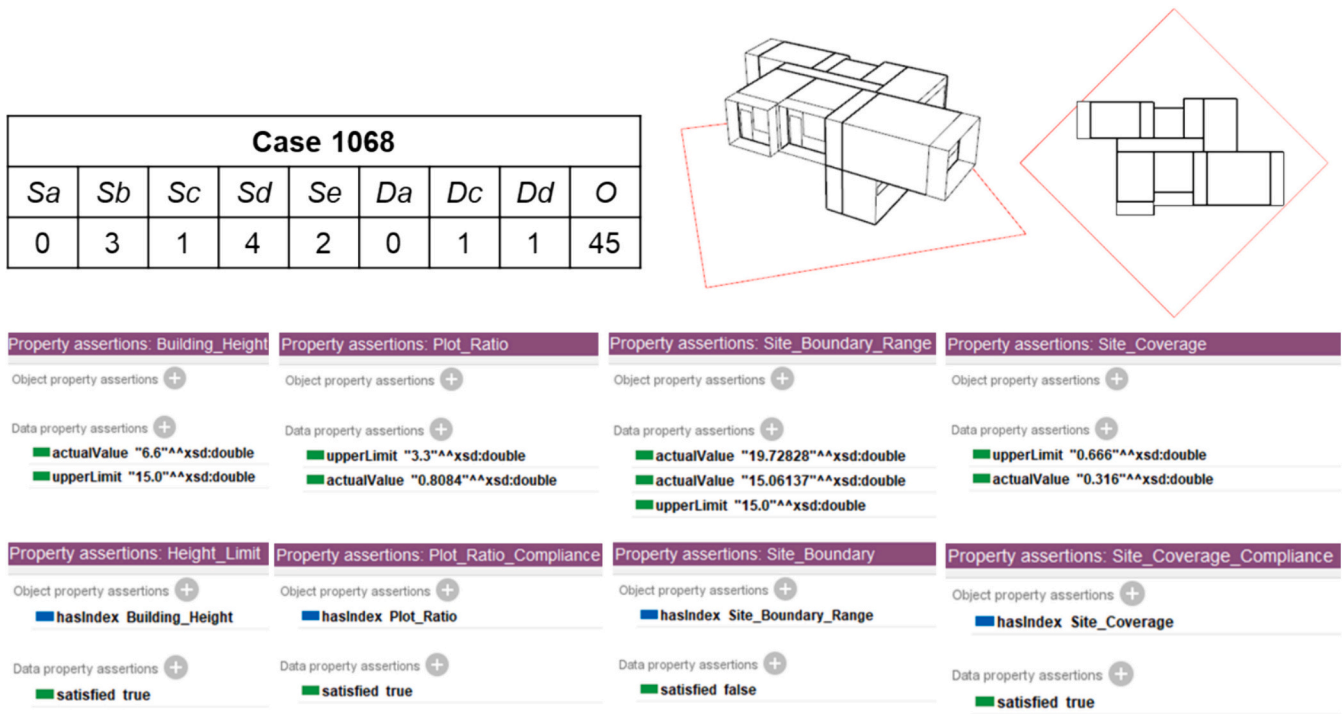


Fig. 14. One case that did not satisfy the requirement for site boundary.

computational time. In contrast to simulation, the strategy-driven reasoning method for performance evaluation belongs to a group of methods with less fidelity but less time consumption. The threshold is the parameter that determines the balance of accuracy and computational time in the proposed knowledge-informed optimization framework. More stringent thresholds could be adopted to accelerate the PGD process if neglecting some Pareto-optimal solutions is acceptable. In addition, the threshold should not be so stringent (e.g., 10% in the illustrative example) that no case can satisfy all the strategies and be selected by the reasoning algorithm. The illustrative example provides a preliminary suggestion on how to choose the threshold. A threshold of 40% is the one that obtains all Pareto-optimal solutions with the least computational time, while 30% is the threshold that can achieve approximately the highest speed while still obtaining an adequate number of Pareto-optimal solutions. To figure out which is the appropriate range of thresholds, further studies with more design cases are needed.

4.4.2. Limitations and future work

Some limitations identified in this study warrant further consideration in future studies.

- (1) As mentioned in Section 4.4.1, the accuracy of the proposed framework relies on the appropriate adoption of strategies. Future studies should address the evaluation of the suitability of the adopted strategies, especially for building design in different climates. Furthermore, the extraction of design strategies from the literature to be translated into machine-readable reasoning algorithms was performed manually. Future research should explore methods for automated extraction of design strategies (as well as regulatory requirements for compliance checking) using natural language processing (NLP) techniques.
- (2) As mentioned in Section 4.4.1, the threshold is the important hyperparameter of the proposed framework. This study provides a preliminary suggestion on the selection of the proper threshold based on the illustrative example, but the universal principles for

threshold selection were not investigated in depth, which remains an issue to be solved in the future.

- (3) In the illustrative example, only one optimization algorithm was implemented, namely, NSGA-II. Studies have shown that other optimization algorithms (e.g., Radial Basis Function Multi-objective Optimization (RBFMOpt) [28]) can outperform NSGA-II. Thus, further research is warranted to investigate the integration of additional meta-heuristic optimization algorithms into the proposed framework.

5. Conclusions

In this study, we introduce a knowledge-informed optimization framework for performance-based generative design (PGD) of sustainable buildings, with the objective of addressing the challenge of long computational time by integrating human domain knowledge of sustainable design strategies into PGD. Firstly, we proposed a PGD-KG schema that effectively captures and represents the topological relations and functionalities within PGD. The PGD-KG schema encompasses five key domains, namely, building, weather, internal heat gain, HVAC system, and regulation and strategy. Secondly, we presented a generation method that automatically produces instance-enriched PGD-KG models from parametric design models with semantic information. Thirdly, we developed cross-domain reasoning algorithms that enable knowledge inference in the PGD-KG models, facilitating automated compliance checking based on regulatory requirements and performance evaluation using sustainable building design strategies. Lastly, we implemented the proposed framework in a design project involving a prefabricated modular detached house to optimize the module layout for minimal cooling energy consumption and maximal daylighting performance. Through experiments, we evaluated the accuracy and computational time performance of the proposed framework. We summarize the major conclusions of this study as follows.

- The proposed knowledge-informed optimization framework is able to find a substantial number of Pareto-optimal solutions ($\geq 87.5\%$ in

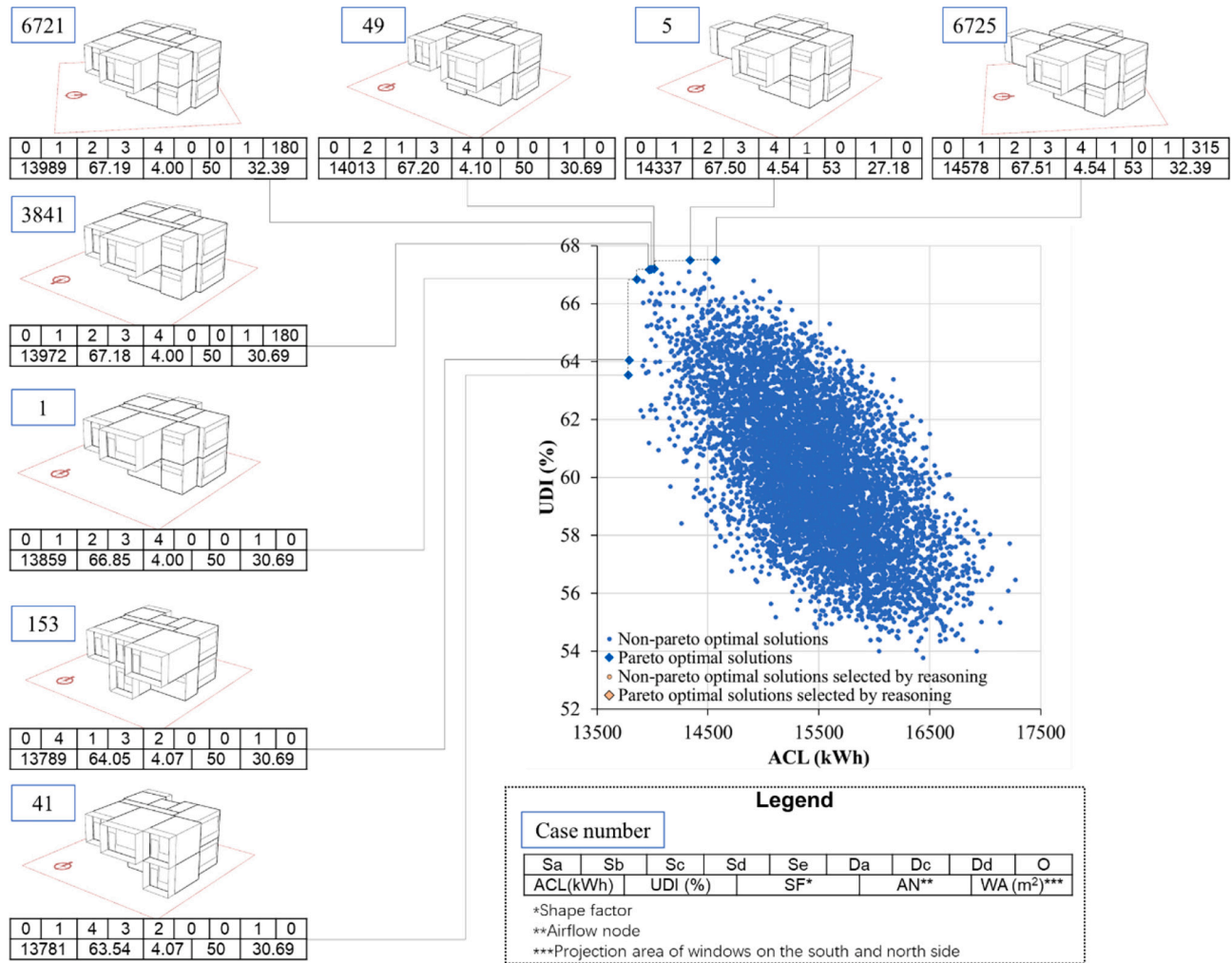


Fig. 15. Solution space and Pareto-optimal solutions of the ground truth.

- the illustrative example) while significantly reducing computational time (by approximately 73% in the illustrative example), compared with the general PGD optimization framework.
- The reduction in computational time in the proposed framework is attributed to the utilization of strategy-driven reasoning algorithms for performance evaluation. These algorithms effectively narrow down the solution space during optimization while ensuring the inclusion of an adequate number of Pareto-optimal solutions ($\geq 87.5\%$ in the illustrative example) by adopting appropriate thresholds.
 - The threshold, which is a hyperparameter determining the scale of cases selected by strategy-driven reasoning algorithms, plays a crucial role in balancing the accuracy and computational time of the

Table 6
Ratio of selected solutions and ratio of selected Pareto-optimal solutions under different thresholds.

Threshold (%)	Ratio of selected solutions (%)	Ratio of selected Pareto-optimal solutions (Selected / ground truth)
75	89.65	8/8
50	47.40	8/8
40	31.04	8/8
30	16.33	7/8
20	11.50	7/8
10	0	0/8

proposed framework. The illustrative example demonstrated that a threshold of 40% is appropriate for obtaining all Pareto-optimal solutions with reduced computational time, while a threshold of 30% is suitable for achieving nearly minimal computational time while still obtaining a satisfactory number of Pareto-optimal solutions.

The major contribution of this study is the successful integration of human domain knowledge, specifically sustainable design strategies, into the optimization process of PGD. To the best of the authors' knowledge, this is the first study to incorporate knowledge integration into PGD. This is done by developing a knowledge graph (KG) model for PGD and implementing KG-based reasoning for case selection through compliance checking and performance evaluation. However, several limitations have been identified. Future studies should focus on addressing these limitations, such as by automating the extraction of design strategies from the literature and translating them into machine-readable reasoning algorithms, exploring universal principles for threshold selection, and integrating additional meta-heuristic optimization algorithms into the proposed framework. In addition, although sustainable performance is focused on in this study, future work could focus on expanding the proposed framework to include other aspects such as aesthetics or structural performance in PGD.

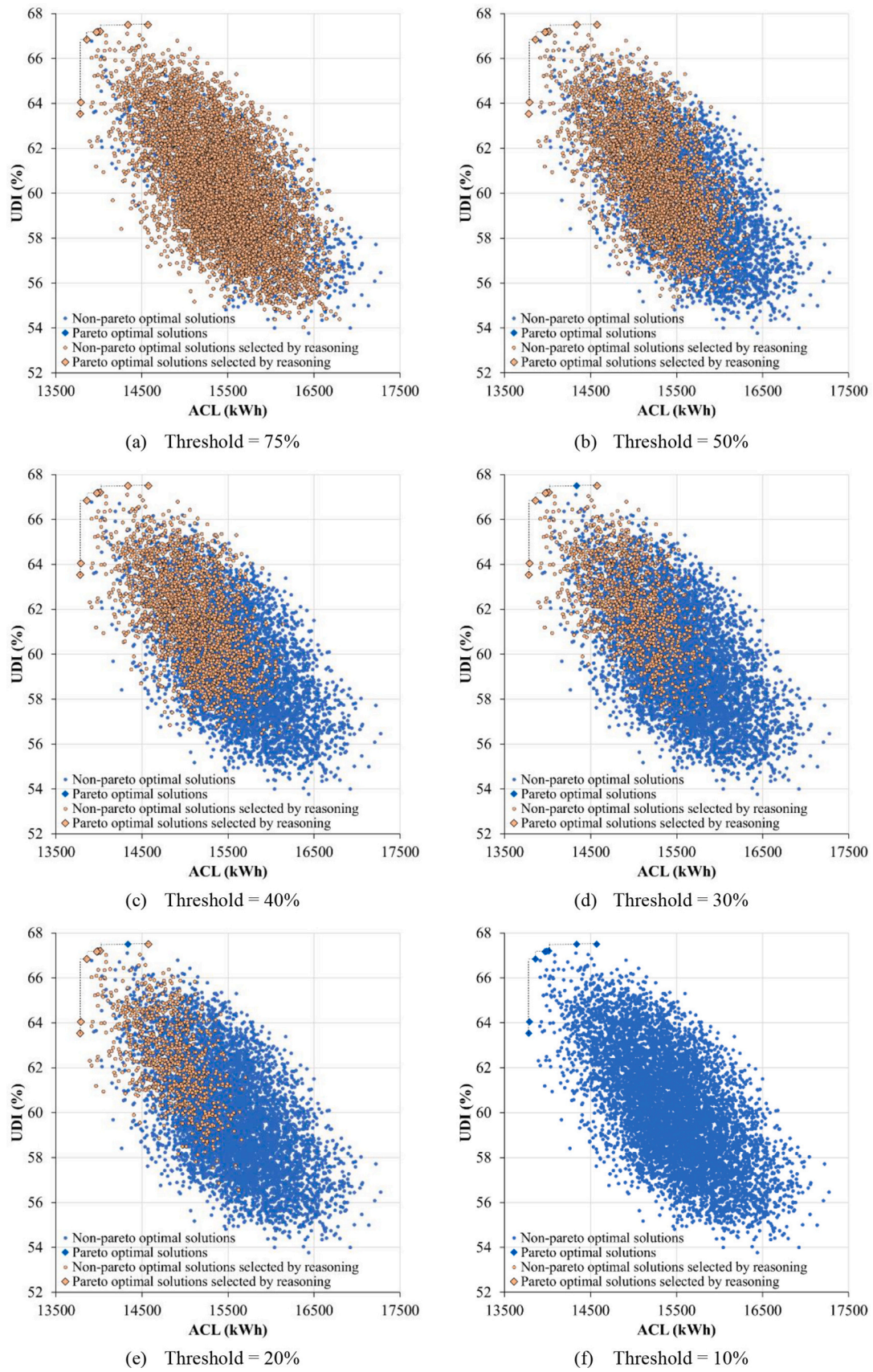


Fig. 16. Selected solution spaces under different thresholds.

Table 7

Accuracy and speed of the knowledge-informed optimization framework under different thresholds.

Threshold (%)	Validity (Selected/ ground truth)	Computational time (s)	Computational time reduction (%) [*]
The general optimization framework			
–	8/8	135,512	–
The knowledge-informed optimization framework			
75	8/8	117,126	13.57
50	8/8	62,837	56.63
40	8/8	51,674	61.87
30	7/8	36,252	73.25
20	7/8	36,243	73.25

^{*} Compared with the time of the general optimization framework.**CRedit authorship contribution statement**

Zhaoji Wu: Writing – original draft, Visualization, Validation, Software, Methodology, Investigation, Formal analysis, Data curation, Conceptualization. **Zhe Wang:** Writing – review & editing, Supervision, Project administration, Methodology, Funding acquisition. **Jack C.P. Cheng:** Writing – review & editing, Supervision, Project administration, Methodology. **Helen H.L. Kwok:** Writing – review & editing, Supervision, Methodology.

Declaration of competing interest

The authors declare that they have no known competing financial

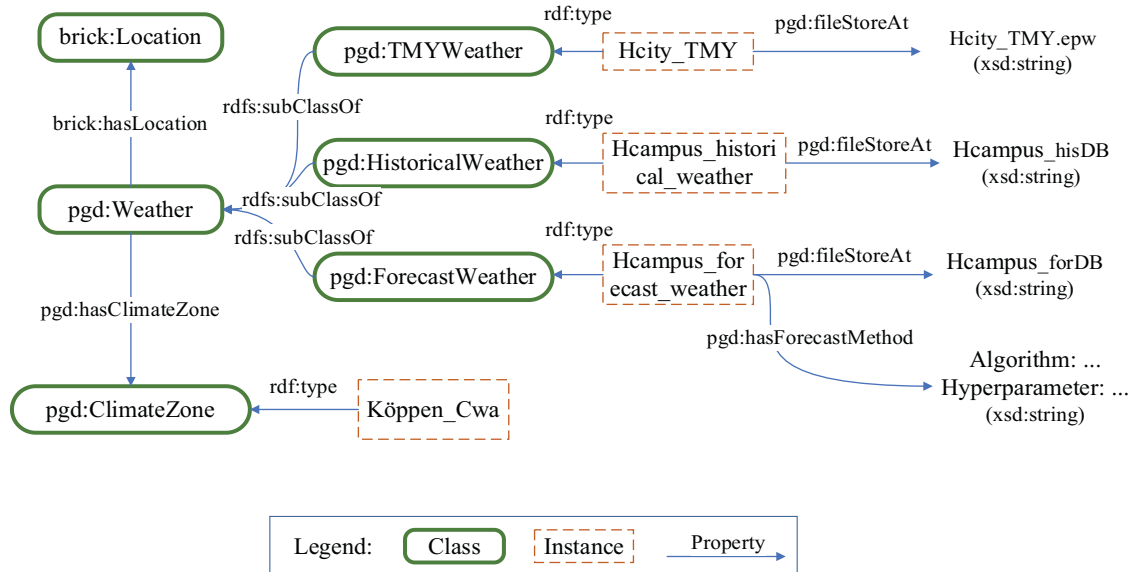
interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Data will be made available on request.

Acknowledgments

This research is supported by the National Science Fund for Excellent Young Scholars (Grant number 52322813). This work is supported in part by the Project of Hetao Shenzhen-Hong Kong Science and Technology Innovation Cooperation Zone (HZQB-KCZYB-2020083).

Appendix A. Components of weather, internal heat gain and HVAC system in the PGD-KG schema**Fig. A.1.** Weather in the PGD-KG schema.

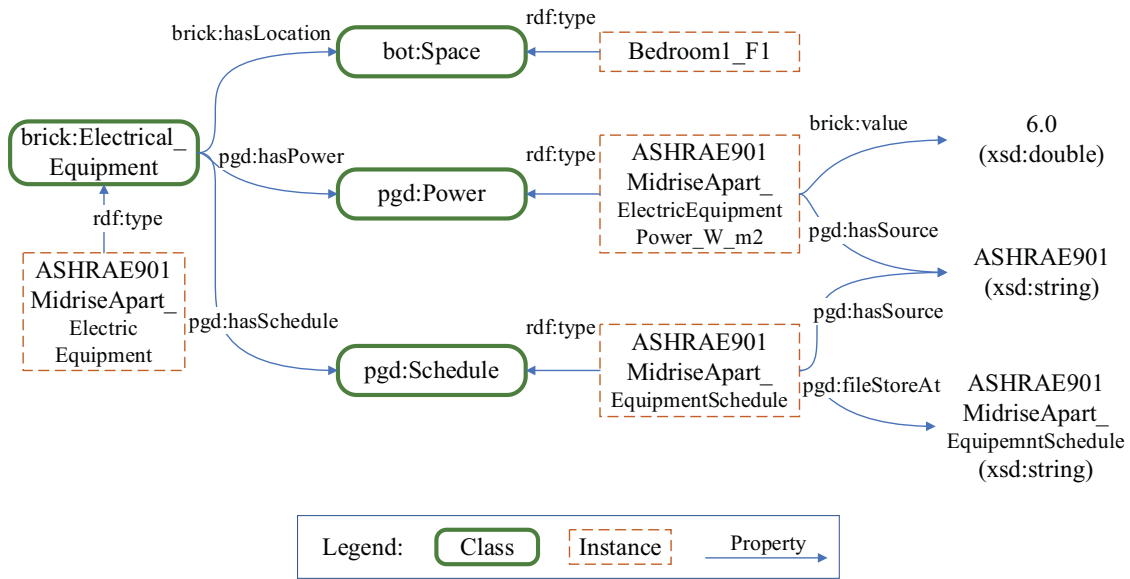


Fig. A.2. Electrical equipment in the PGD-KG schema.

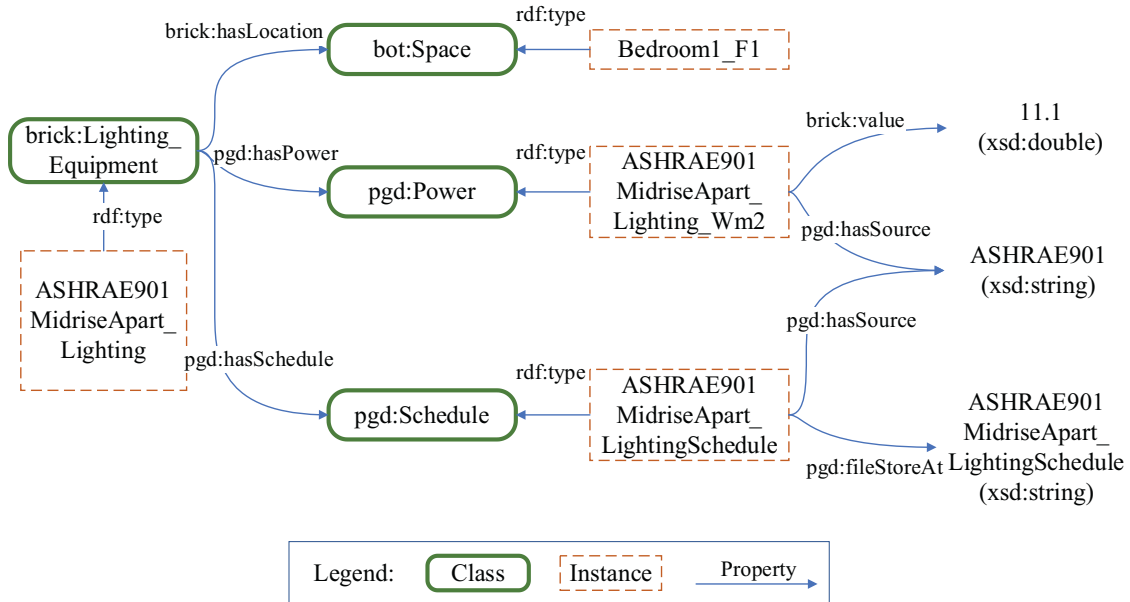


Fig. A.3. Lighting equipment in the PGD-KG schema.

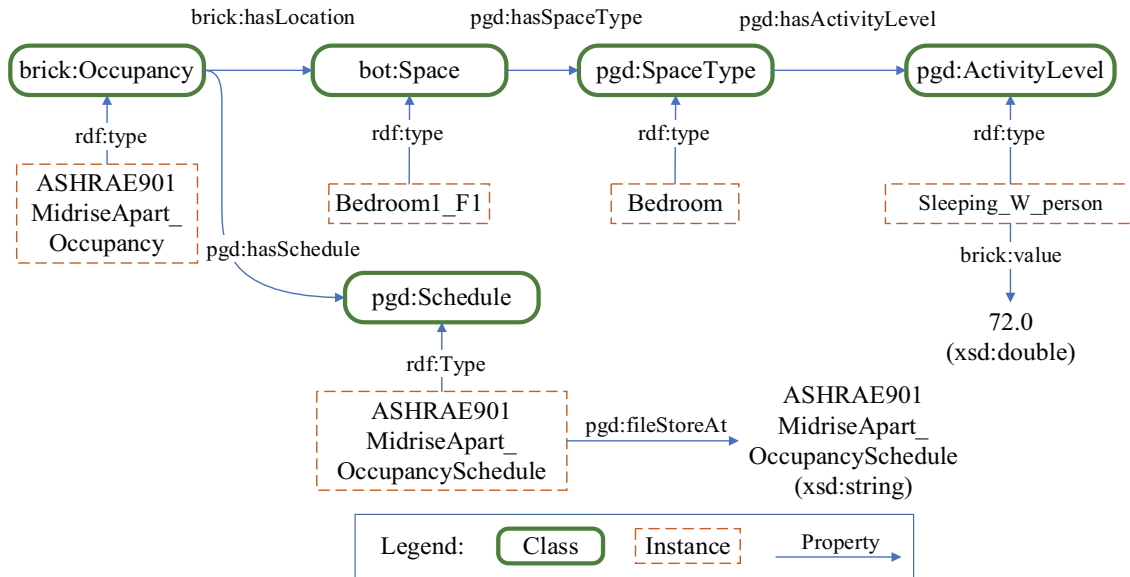


Fig. A.4. Occupancy in the PGD-KG schema.

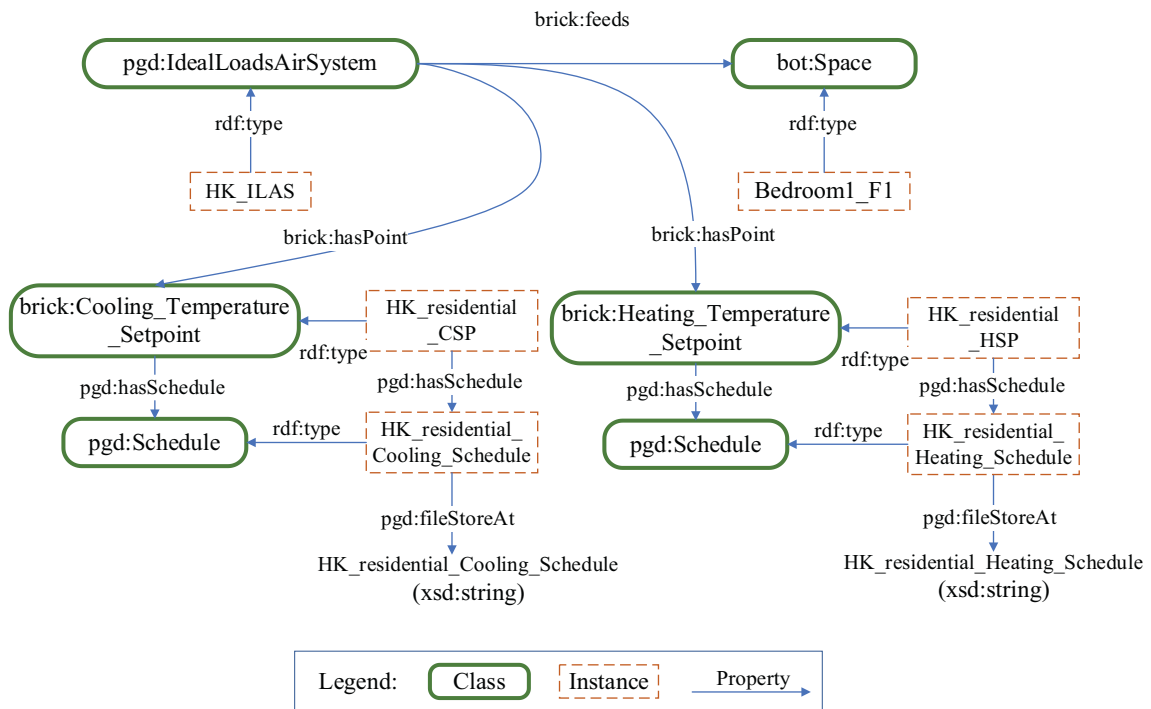


Fig. A.5. HVAC system in the PGD-KG schema.

Appendix B. Reasoning algorithms

Algorithm 3. (in SPARQL): Reasoning for the number of walls that a fenestration surface is attached to.

```

1      SELECT ?fenestration (COUNT(?wall) AS ?count)
2      WHERE {?wall rdf:type bot:Element .
3              ?fenestration rdf:type bot:Element .
4              ?wall bot:hasSubElement ?fenestration .
5              }
6      GROUP BY ?fenestration

```

Algorithm 4. (in SWRL): Reasoning for airflow topology relations.

```

1      //Airflow between a space and a fenestration surface
2      bot:Space(?space)
3      ^bot:hasElement(?space,?buildingsurface)
4      ^bot:hasSubElement(?buildingsurface,?fenestration)
5      ->pgd:hasAirFlow(?space,?fenestration)
6      //Airflow between a space and an air wall
7      bot:Space(?space)
8      ^bot:hasElement(?space,?airwall)
9      ^pgd:hasSurfaceType(?a"rwall,""irWall")
10     ->pgd:hasAirFlow(?space,?airwall)
11     //Airflow between an air wall and its adjacent air wall
12     bot:Element(?airwall1)
13     ^bot:Element(?airwall2)
14     ^pgd:hasSurfaceType(?ai"wall1,""irWall")
15     ^pgd:hasSurfaceType(?ai"wall2,""irWall")
16     ^bot:adjacentElement(?airwall1,?airwall2)
17     ->pgd:hasAirFlow(?airwall1,?airwall2)

```

Algorithm 5. (in SPARQL): Reasoning for actual height of a building.

```

1  SELECT (MAX(?maxValue) AS ?overallMaxValue)
2  WHERE {
3      {
4          SELECT (MAX(?value1) AS ?maxValue)
5          WHERE {
6              ?surface rdf:type bot:Element .
7              ?surface pgd:hasVertex1Z_m ?value1 .
8          }
9      } UNION
10     {
11         SELECT (MAX(?value2) AS ?maxValue)
12         WHERE {
13             ?surface rdf:type bot:Element .
14             ?surface pgd:hasVertex2Z_m ?value2 .
15         }
16     } UNION
17     {
18         SELECT (MAX(?value3) AS ?maxValue)
19         WHERE {
20             ?surface rdf:type bot:Element .
21             ?surface pgd:hasVertex3Z_m ?value3 .
22         }
23     } UNION
24     {
25         SELECT (MAX(?value4) AS ?maxValue)
26         WHERE {
27             ?surface rdf:type bot:Element .
28             ?surface pgd:hasVertex4Z_m ?value4 .
29         }
30     }
31 }

```

Obtain the maximums of the z coordinates of the first vertex.

Obtain the maximums of the z coordinates of the second vertex.

Obtain the maximums of the z coordinates of the third vertex.

Obtain the maximums of the z coordinates of the fourth vertex.

Algorithm 6. (in SPARQL): Reasoning for actual percentage of site coverage.

```

1  SELECT ?siteCoverage
2  WHERE {
3      ?site rdf:type bot:Site .
4      ?site pgd:has Area_m2 ?siteArea .
5      {
6          SELECT (SUM(?GFFloorArea) AS ?sumGFFloorArea)
7          WHERE {
8              {GF} bot:hasSpace ?space .
9              ?space bot:hasElement ?floor .
10             ?floor pgd:hasSurfaceType "Floor" .
11             ?floor pgd:hasFloorArea_m2 ?GFFloorArea.
12         }
13     }
14     BIND((?sumGFFloorArea/?siteArea) AS ?siteCoverage)
15 }

```

*{GF}: the Uniform Resource Identifier (URI) of the ground floor.

Algorithm 7. (in SPARQL): Reasoning for actual plot ratio.

```

1  SELECT ?plotRatio
2  WHERE {
3      ?site rdf:type bot:Site .
4      ?site pgd:has Area_m2 ?siteArea .
5      {
6          SELECT (SUM(?floorArea) AS ?sumFloorArea)
7          WHERE {
8              ?floor rdf:type bot:Element .
9              ?floor pgd:hasSurfaceType "Floor" .
10             ?floor pgd:hasFloorArea_m2 ?floorArea .
11         }
12     }
13     BIND((?sumFloorArea/?siteArea) AS ?plotRatio) .
14 }

```

Algorithm 8. (in SPARQL): Reasoning for actual site boundary.

```

1  SELECT ?transValuexRange ?transValueyRange
2  WHERE {
3      {SELECT (MAX(?transValuex) AS ?OVERALLMAXtransValuex)
4          (MAX(?transValuey) AS ?OVERALLMAXtransValuey)
5          (MIN(?transValuex) AS ?OVERALLMINtransValuex)
6          (MIN(?transValuey) AS ?OVERALLMINtransValuey)
7      WHERE {
8          {SELECT ?transValuex ?transValuey
9              WHERE {
10                 ?surface rdf:type bot:Element .
11                 ?surface pgd:hasVertex1X_m ?value1x .
12                 ?surface pgd:hasVertex1Y_m ?value1y .
13                 BIND(?value1x*{cosA}+?value1y*{sinA} AS ?transValuex) .
14                 BIND(-?value1x*{sinA}+?value1y*{cosA} AS ?transValuey) .
15             }
16         } UNION
17         {SELECT ?transValuex ?transValuey
18             WHERE {
19                 ?surface rdf:type bot:Element .
20                 ?surface pgd:hasVertex2X_m ?value2x .
21                 ?surface pgd:hasVertex2Y_m ?value2y .
22                 BIND(?value2x*{cosA}+?value2y*{sinA} AS ?transValuex) .
23                 BIND(-?value2x*{sinA}+?value2y*{cosA} AS ?transValuey) .
24             } UNION
25             {SELECT ?transValuex ?transValuey
26                 WHERE {
27                     ?surface rdf:type bot:Element .
28                     ?surface pgd:hasVertex3X_m ?value3x .
29                     ?surface pgd:hasVertex3Y_m ?value3y .
30                     BIND(?value3x*{cosA}+?value3y*{sinA} AS ?transValuex) .
31                     BIND(-?value3x*{sinA}+?value3y*{cosA} AS ?transValuey) .
32                 }
33             } UNION
34             {SELECT ?transValuex ?transValuey
35                 WHERE {
36                     ?surface rdf:type bot:Element .
37                     ?surface pgd:hasVertex4X_m ?value4x .
38                     ?surface pgd:hasVertex4Y_m ?value4y .
39                     BIND(?value4x*{cosA}+?value4y*{sinA} AS ?transValuex) .
40                     BIND(-?value4x*{sinA}+?value4y*{cosA} AS ?transValuey) .
41                 }
42             }
43         }
44     }
45     BIND(?OVERALLMAXtransValuex-?OVERALLMINtransValuex AS ?transValuexRange) .
46     BIND(?OVERALLMAXtransValuey-?OVERALLMINtransValuey AS ?transValueyRange) .
47 }

```

Calculate the maximal x, minimal x, maximal y and minimal y coordinates of the vertices (the final step).

Retrieve the x and y coordinates of the first vertex and conduct coordinate transformation from the building coordinate to the true north coordinate.

Retrieve the x and y coordinates of the second vertex and conduct coordinate transformation from the building coordinate to the true north coordinate.

Retrieve the x and y coordinates of the third vertex and conduct coordinate transformation from the building coordinate to the true north coordinate.

Retrieve the x and y coordinates of the fourth vertex and conduct coordinate transformation from the building coordinate to the true north coordinate.

Obtain the range of the x and y coordinates.

*{sinA}: the sine of the orientation, {cosA}: the cosine of the orientation.

Algorithm 9. (in SWRL): Reasoning for compliance assessment.

```

1 //if the actual value is within the limit, the compliance of the constraint is true.
2 pgd:Constraint(?constraint)
3 ^ pgd:Index(?index)
4 ^ pgd:hasIndex(?constraint, ?index)
5 ^ pgd:upperLimit(?index, ?limitvalue)
6 ^ pgd:actualValue(?index, ?actualvalue)
7 ^ swrlb:lessThanOrEqual(?actualvalue, ?limitvalue)
8 -> pgd:satisfied(?constraint, true)
9 //if the actual value is beyond the limit, the compliance of the constraint is false.
10 pgd:Constraint(?constraint)
11 ^ pgd:Index(?index)
12 ^ pgd:hasIndex(?constraint, ?index)
13 ^ pgd:upperLimit(?index, ?limitvalue)
14 ^ pgd:actualValue(?index, ?actualvalue)
15 ^ swrlb:greaterThan(?actualvalue, ?limitvalue)
16 -> pgd:satisfied(?constraint, false)

```

Algorithm 10. (in SPARQL): Reasoning for the shape factor.

```

1 SELECT ?shapefactor
2 WHERE {
3     {SELECT (SUM(?outdoorarea) AS ?totaloutdoorarea) (SUM(?volume) AS ?totalvolume)
4     WHERE {?space rdf:type bot:Space .
5           ?space pgd:hasVolume_m3 ?volume .
6           {SELECT (SUM(?area) AS ?outdoorarea)
7           WHERE {?surface rdf:type bot:Element .
8                 ?surface pgd:hasSurfaceType "Wall" .
9                 ?surface pgd:hasOutsideBoundaryCondition "Outdoors" .
10                ?surface pgd:hasArea_m2 ?area .
11            }
12        } UNION
13        {SELECT (SUM(?area) AS ?outdoorarea)
14        WHERE {?surface rdf:type bot:Element .
15              ?surface pgd:hasSurfaceType "Roof" .
16              ?surface pgd:hasOutsideBoundaryCondition "Outdoors" .
17              ?surface pgd:hasArea_m2 ?area .
18          }
19      } UNION
20      {SELECT (SUM(?area) AS ?outdoorarea)
21      WHERE {?surface rdf:type bot:Element .
22            ?surface pgd:hasSurfaceType "Floor" .
23            ?surface pgd:hasOutsideBoundaryCondition "Outdoors" .
24            ?surface pgd:hasArea_m2 ?area .
25        }
26    }
27 }
28 }
29 BIND((?totaloutdoorarea/?totalvolume) AS ?shapefactor)
30 }

```

Calculate the total area and the building volume.

Calculate the area of the outdoor walls.

Calculate the area of the outdoor roofs.

Calculate the area of the outdoor floors.

Calculate the shape factor.

Algorithm 11. (in SPARQL): Reasoning for the number of nodes in overall airflow path.

1	SELECT (COUNT(DISTINCT ?intermediateNode) AS ?nodeCount)	
2	WHERE {	
3	?window1 rdf:type bot:Element .	} Retrieve the first outdoor window.
4	?window1 pgd:hasSurfaceType "Window" .	
5	?window1 pgd:hasOutsideBoundaryCondition "Outdoors" .	
6	?window2 rdf:type bot:Element .	} Retrieve the second outdoor window.
7	?window2 pgd:hasSurfaceType "Window" .	
8	?window2 pgd:hasOutsideBoundaryCondition "Outdoors" .	
9	?window1 pgd:hasAirFlow* ?intermediateNode .	} Calculate the number of nodes between the two windows.
10	?intermediateNode pgd:hasAirFlow* ?window2 .	
11	}	

Algorithm 12. (in Python): Reasoning for projection area of windows on the south and north side.

1	query_south_north_window_projection_area = ""	} Retrieve the areas of windows on the south and north side.
2	SELECT ?area ?nvx ?nvy	
3	WHERE {?window rdf:type bot:Element .	
4	?window pgd:hasSurfaceType "Window" .	
5	?window pgd:hasOutsideBoundaryCondition "Outdoors" .	
6	?window pgd:hasArea_m2 ?area .	
7	?window pgd:hasNormalVectorX_m ?nvx .	
8	?window pgd:hasNormalVectorY_m ?nvy .	
9	}	
10	""	} Calculate the projected area.
11	qres_south_north_window_projection_area = g.query(query_south_north_window_projection_area)	
12	south_north_window_projection_area = 0	
13	east_west_window_projection_area = 0	
14	for row in qres_south_north_window_projection_area:	
15	window_area = float(row[0])	
16	normal_vector_x = float(row[1])	
17	normal_vector_y = float(row[2])	
18	south_north_projected_area = window_area * pow(1 - pow(normal_vector_x * cosA + normal_vector_y * sinA, 2), 0.5)	
19	south_north_window_projection_area += south_north_projected_area	
20	east_west_projected_area = window_area * pow(1 - pow(normal_vector_y * cosA - normal_vector_x * sinA, 2), 0.5)	
21	east_west_window_projection_area += east_west_projected_area	

Appendix C. Simulation and optimization settings of the illustrative example**Table C.1**

Construction and material settings.

Construction	Material	Parameter
Wall	Layer 1: 1IN stucco	Roughness: Smooth, Thickness (m): 0.0253, Conductivity (W/m·K): 0.691, Density (kg/m ³): 1858, Specific heat (J/kg·K): 836, Thermal absorptance: 0.9, Solar absorptance: 0.7, Visible absorptance: 0.92.
	Layer 2: 8IN concrete	Roughness: Rough, Thickness (m): 0.2032, Conductivity (W/m·K): 1.310, Density (kg/m ³): 2240, Specific heat (J/kg·K): 836, Thermal absorptance: 0.9, Solar absorptance: 0.7, Visible absorptance: 0.7.

(continued on next page)

Table C.1 (continued)

Construction	Material	Parameter
Roof	Layer 3: Typical insulation-R5	Roughness: MediumSmooth, R-value ($\text{m}^2\cdot\text{K}/\text{W}$): 0.88, Thermal absorptance: 0.9, Solar absorptance: 0.7, Visible absorptance: 0.7.
	Layer 4: 1/2IN gypsum	Roughness: Smooth, Thickness (m): 0.0127, Conductivity ($\text{W}/\text{m}\cdot\text{K}$): 0.160, Density (kg/m^3): 785, Specific heat ($\text{J}/\text{kg}\cdot\text{K}$): 829, Thermal absorptance: 0.9, Solar absorptance: 0.4, Visible absorptance: 0.4.
	Layer 1: Roof membrane - highly reflective	Roughness: VeryRough, Thickness (m): 0.0095, Conductivity ($\text{W}/\text{m}\cdot\text{K}$): 0.160, Density (kg/m^3): 1121, Specific heat ($\text{J}/\text{kg}\cdot\text{K}$): 1459, Thermal absorptance: 0.75, Solar absorptance: 0.45, Visible absorptance: 0.7.
	Layer 2: Typical insulation-R26	Roughness: MediumSmooth, R-value ($\text{m}^2\cdot\text{K}/\text{W}$): 4.58, Thermal absorptance: 0.9, Solar absorptance: 0.7, Visible absorptance: 0.7.
Floor	Layer 3: Metal roof surface	Roughness: Smooth, Thickness (m): 0.0008, Conductivity ($\text{W}/\text{m}\cdot\text{K}$): 45.2, Density (kg/m^3): 7824, Specific heat ($\text{J}/\text{kg}\cdot\text{K}$): 500, Thermal absorptance: 0.9, Solar absorptance: 0.7, Visible absorptance: 0.7.
	Layer 1: Typical insulation R8	Roughness: MediumSmooth, R-value ($\text{m}^2\cdot\text{K}/\text{W}$): 1.41, Thermal absorptance: 0.9, Solar absorptance: 0.7, Visible absorptance: 0.7.
	Layer 2: Normal weight concrete floor	Roughness: MediumRough, Thickness (m): 0.1016, Conductivity ($\text{W}/\text{m}\cdot\text{K}$): 2.31, Density (kg/m^3): 2322, Specific heat ($\text{J}/\text{kg}\cdot\text{K}$): 831, Thermal absorptance: 0.9, Solar absorptance: 0.7, Visible absorptance: 0.7.
	Layer 3: Typical carpet pad	Roughness: VeryRough, R-value ($\text{m}^2\cdot\text{K}/\text{W}$): 0.216, Thermal absorptance: 0.9, Solar absorptance: 0.7, Visible absorptance: 0.8.
Door	Layer 1: F08 metal surface	Roughness: Smooth, Thickness (m): 0.0008, Conductivity ($\text{W}/\text{m}\cdot\text{K}$): 45.2, Density (kg/m^3): 7824, Specific heat ($\text{J}/\text{kg}\cdot\text{K}$): 500, Thermal absorptance: 0.9, Solar absorptance: 0.7, Visible absorptance: 0.7.
	Layer 2: Typical insulation-R3	Roughness: MediumSmooth, R-value ($\text{m}^2\cdot\text{K}/\text{W}$): 0.528, Thermal absorptance: 0.9, Solar absorptance: 0.7, Visible absorptance: 0.7.
Window	Layer 1: U 0.45 SHGC 0.25 simple glazing	U-factor ($\text{W}/\text{m}^2\cdot\text{K}$): 2.5551, SHGC: 0.25, VT: 0.6.

Table C.2

Internal heat gain settings.

Internal heat gain	Power density (W/m ²)	Power (W)	People per area (people/m ²)
Lighting	11.1	–	–
Equipment	6	–	–
Occupant	–	95	0.025

Table C.3

Schedule settings.

Hour	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24
Equipment	0.45	0.41	0.39	0.38	0.38	0.43	0.54	0.65	0.66	0.67	0.69	0.7	0.69	0.66	0.65	0.68	0.8	1	1	0.93	0.89	0.85	0.71	0.58
Light	0.067	0.067	0.067	0.067	0.187	0.394	0.44	0.393	0.172	0.119	0.119	0.119	0.119	0.119	0.119	0.206	0.439	0.616	0.829	0.986	1	0.692	0.384	0.16
Occupant	1	1	1	1	1	1	1	0.85	0.39	0.25	0.25	0.25	0.25	0.25	0.25	0.25	0.3	0.52	0.87	0.87	1	1	1	1

Table C.4

Parameter settings of NSGA-II algorithm in the illustrative example.

Population size	Mutation probability	Crossover probability	Mutation distribution index	Crossover distribution index
100	0.1	0.9	20	20

Appendix D. PGD-KG model files

The PGD-KG models mentioned in the main text can be found by this link: https://github.com/GeorgeZWu/PGD_KG_Schema.

References

- [1] United Nations Environment Programme. Global status report for buildings and construction, 2022. <https://www.unep.org/resources/publication/2022-global-status-report-buildings-and-construction>; 2022. accessed October 31, 2023.
- [2] Davis D. The MacLeamy curve. <https://www.danieldavis.com/macleamy/>; 2011. accessed October 31, 2023.
- [3] Zhang L, Feng G, Li A, Huang K, Chang S. Comprehensive evaluation and analysis of a nearly zero-energy building heating system using a multi-source heat pump in severe cold region. In: Building simulation. Springer; 2023. p. 1–22.
- [4] Ekici B, Cubukcuoglu C, Turrin M, Sariyildiz IS. Performative computational architecture using swarm and evolutionary optimisation: a review. Build Environ 2019;147:356–71. <https://doi.org/10.1016/j.buildenv.2018.10.023>.
- [5] Pacheco R, Ordóñez J, Martínez G. Energy efficient design of building: a review. Renew Sust Energ Rev 2012;16:3559–73. <https://doi.org/10.1016/j.rser.2012.03.045>.
- [6] An J, Wu Y, Gui C, Yan D. Chinese prototype building models for simulating the energy performance of the nationwide building stock. Build Simul 2023;16: 1559–82. <https://doi.org/10.1007/s12273-023-1058-5>.
- [7] Wang W, Rivard H, Zmeureanu R. An object-oriented framework for simulation-based green building design optimization with genetic algorithms. Adv Eng Inform 2005;19:5–23. <https://doi.org/10.1016/j.aei.2005.03.002>.
- [8] Machairas V, Tsangrassoulis A, Axarli K. Algorithms for optimization of building design: a review. Renew Sust Energ Rev 2014;31:101–12. <https://doi.org/10.1016/j.rser.2013.11.036>.
- [9] Su Z, Yan W. A fast genetic algorithm for solving architectural design optimization problems. AIEDAM 2015;29:457–69. <https://doi.org/10.1017/S089006041500044X>.
- [10] Brown G, DeKay M. Sun, wind, and light. 2nd ed. New York, USA: Wiley; 2013.
- [11] Heywood H. 101 rules of thumb for low energy architecture. London, UK: Riba publishing; 2019.
- [12] Kwok AG, Grondzik W. The green studio handbook: Environmental strategies for schematic design. Oxfordshire, UK: Routledge; 2018.
- [13] Ji S, Pan S, Cambria E, Marttinen P, Yu PS. A survey on knowledge graphs: representation, acquisition, and applications. IEEE Trans Neural Netw Learn Syst 2022;33:494–514. <https://doi.org/10.1109/TNNLS.2021.3070843>.
- [14] Pauwels P, Zhang S, Lee Y-C. Semantic web technologies in AEC industry: a literature overview. Autom Constr 2017;73:145–65. <https://doi.org/10.1016/j.autcon.2016.10.003>.
- [15] Herr CM, Kvan T. Adapting cellular automata to support the architectural design process. Autom Constr 2007;16:61–9. <https://doi.org/10.1016/j.autcon.2005.10.005>.
- [16] Lakhdari K, Sriti L, Painter B. Parametric optimization of daylight, thermal and energy performance of middle school classrooms, case of hot and dry regions. Build Environ 2021;204:108173. <https://doi.org/10.1016/j.buildenv.2021.108173>.
- [17] Mohammad S, Saryazdi E, Etemad A, Shafaat A, Bahman AM. Data-driven performance analysis of a residential building applying artificial neural network (ANN) and multi-objective genetic algorithm (GA). Build Environ 2022;225: 109633. <https://doi.org/10.1016/j.buildenv.2022.109633>.
- [18] Attia S, Hamdy M, O'Brien W, Carlucci S. Assessing gaps and needs for integrating building performance optimization tools in net zero energy buildings design. Energy Buildings 2013;60:110–24. <https://doi.org/10.1016/j.enbuild.2013.01.016>.
- [19] Ramallo-González AP, Coley DA. Using self-adaptive optimisation methods to perform sequential optimisation for low-energy building design. Energy Buildings 2014;81:18–29. <https://doi.org/10.1016/j.enbuild.2014.05.037>.
- [20] Choudhary R, Malkawi A, Papalambros PY. A hierarchical design optimization framework for building performance analysis. In: Proceedings of the 8-Th IBPSA conference, Eindhoven, Netherlands; 2003.
- [21] Westermann P, Evins R. Surrogate modelling for sustainable building design – a review. Energy Buildings 2019;198:170–86. <https://doi.org/10.1016/j.enbuild.2019.05.057>.
- [22] Magnier L, Haghighat F. Multiobjective optimization of building design using TRNSYS simulations, genetic algorithm, and artificial neural network. Build Environ 2010;45:739–46. <https://doi.org/10.1016/j.buildenv.2009.08.016>.
- [23] Wang W, Zmeureanu R, Rivard H. Applying multi-objective genetic algorithms in green building design optimization. Build Environ 2005;40:1512–25.
- [24] Asl MR, Zarrinmehr S, Bergin M, Yan W. BPOpt: a framework for BIM-based performance optimization. Energy Buildings 2015;108:401–12.
- [25] Ekici B, Chatzikonstantinou I, Sariyildiz S, Tasgetiren MF, Pan Q-K. A multi-objective self-adaptive differential evolution algorithm for conceptual high-rise building design. In: 2016 IEEE congress on evolutionary computation (CEC). Vancouver, BC: IEEE; 2016. p. 2272–9. <https://doi.org/10.1109/CEC.2016.7744069>.
- [26] Zhang R, Waibel C, Wortmann T. Aerodynamic shape optimization for high-rise conceptual design. In: Proceedings of the eCAADe; 2020.
- [27] Zhu L, Wang B, Sun Y. Multi-objective optimization for energy consumption, daylighting and thermal comfort performance of rural tourism buildings in North China. Build Environ 2020;176:106841. <https://doi.org/10.1016/j.buildenv.2020.106841>.
- [28] Zhang R, Xu X, Zhai P, Liu K, Kong L, Wang W. Agile and integrated workflow proposal for optimising energy use, solar and wind energy potential, and structural stability of high-rise buildings in early design decisions. Energy Buildings 2023;300: 113692. <https://doi.org/10.1016/j.enbuild.2023.113692>.
- [29] Ji Y, Wang W, He Y, Li L, Zhang H, Zhang T. Performance in generation: An automatic generalizable generative-design-based performance optimization framework for sustainable building design. Energy Buildings 2023;298:113512. <https://doi.org/10.1016/j.enbuild.2023.113512>.
- [30] Benincá L, Crespo Sánchez E, Passuello A, Karini Leitzke R, Grala da Cunha E, Barroso J Maria González. Multi-objective optimization of the solar orientation of two residential multifamily buildings in South Brazil. Energy Buildings 2023;285: 112838. <https://doi.org/10.1016/j.enbuild.2023.112838>.
- [31] Lygerakis F, Kampelis N, Kolokotsa D. Knowledge Graphs' ontologies and applications for energy efficiency in buildings: a review. Energies 2022;15:7520. <https://doi.org/10.3390/en15207520>.
- [32] Pauwels P, Terkaj W. EXPRESS to OWL for construction industry: towards a recommendable and usable ifcOWL ontology. Autom Constr 2016;63:100–33. <https://doi.org/10.1016/j.autcon.2015.12.003>.
- [33] Rasmussen MH, Lefrançois M, Schneider GF, Pauwels P. BOT: the building topology ontology of the W3C linked building data group. Semantic Web 2020;12: 143–61. <https://doi.org/10.3233/SW-200385>.
- [34] Balaji B, Bhattacharya A, Fierro G, Gao J, Gluck J, Hong D, et al. Brick : metadata schema for portable smart building applications. Appl Energy 2018;226:1273–92. <https://doi.org/10.1016/j.apenergy.2018.02.091>.
- [35] Pauwels P, Corry E, O'Donnell J. Representing SimModel in the web ontology language. In: Computing in civil and building engineering. 2014. Orlando, Florida, United States: American Society of Civil Engineers; 2014. p. 2271–8. <https://doi.org/10.1061/9780784413616.282>.
- [36] Wu Z, Cheng JCP, Wang Z, Kwok HHL. An ontology-based framework for automatic building energy modeling with thermal zoning. Energy Buildings 2023; 296:113267. <https://doi.org/10.1016/j.enbuild.2023.113267>.
- [37] Goldberg DE. Genetic algorithms in search, optimization and machine learning. 1st ed. USA: Addison-Wesley Longman Publishing Co., Inc; 1989.
- [38] Eberhart R, Kennedy J. A new optimizer using particle swarm theory. In: MHS'95. Proceedings of the sixth international symposium on micro machine and human science. IEEE; 1995. p. 39–43.
- [39] OWL Working Group, PrimerExampleTurtle - OWL. <https://www.w3.org/2007/OWL/wiki/PrimerExampleTurtle>; 2008. accessed October 31, 2023.
- [40] U.S. Department of Energy. Tips and tricks for using EnergyPlus. https://energypplus.net/assets/nrel_custom/pdfs/pdfs_v22.1.0/TipsAndTricksUsingEnergyPlus.pdf; 2022. accessed October 31, 2023.
- [41] Davidson S. Grasshopper - algorithmic modeling for Rhino. <https://www.grasshopper3d.com/>; 2023. accessed October 31, 2023.
- [42] Autodesk Inc. Learn - dynamo BIM. <https://dynamobim.org/>; 2021.
- [43] Negendahl K. Building performance simulation in the early design stage: An introduction to integrated dynamic models. Autom Constr 2015;54:39–53. <https://doi.org/10.1016/j.autcon.2015.03.002>.
- [44] Ladybug Tools LLC. Intersect solids - honeybee primer, intersect solids - honeybee primer. https://docs.ladybugtools.com/honeybee-primer/components/0_create/intersect_solids; 2023.
- [45] World Wide Web Consortium. SPARQL 1.1 query language. <https://www.w3.org/TR/sparql11-query/>; 2023.
- [46] Horrocks I, Patel-Schneider PF, Boley H, Tabet S, Groszof B, Dean M, et al. SWRL: a semantic web rule language combining OWL and RuleML, W3C member submission21; 2004. p. 1–31.
- [47] Dimyadi J, Clifton C, Spearpoint M, Amor R. Regulatory knowledge encoding guidelines for automated compliance audit of building engineering design. In: Computing in civil and building engineering; 2014. p. 536–43. <https://doi.org/10.1061/9780784413616.067>.

- [48] Buildings Department. The government of the Hong Kong special administrative region, building (planning) regulations. <https://www.elegislation.gov.hk/hk/cap123F>; 2012.
- [49] Robert McNeel & Associates. rhino3dm-PyPI. <https://pypi.org/project/rhino3dm/>; 2022. accessed October 31, 2023.
- [50] RDFLib Team. rdflib 6.2.0. <https://rdflib.readthedocs.io/>; 2022. accessed May 18, 2023.
- [51] U.S. Department of Energy's (DOE) Building Technologies Office (BTO), National Renewable Energy Laboratory (NREL). EnergyPlus. <https://energyplus.net/>; 2023. accessed October 31, 2023.
- [52] Fuller D, McNeil A. Radiance - Radsite. <https://www.radiance-online.org/>; 2017. accessed October 31, 2023.
- [53] Nabil A, Mardaljevic J. Useful daylight illuminances: a replacement for daylight factors. *Energ Buildings* 2006;38:905–13.
- [54] American Society of Heating, Refrigerating and Air-Conditioning Engineers, Inc. ANSI/ASHRAE/IES Standard 90.1: energy standard for buildings except low-rise residential buildings. Atlanta, GA, USA: ASHRAE; 2022.
- [55] Kam M. 25.5 deg C and human comfort. Report Energy and Efficiency Office, the Government of Hong Kong Special Administrative Region. 2004.
- [56] Zhang Y. Design criteria of built thermal environment for hot summer & warm winter zone of China. *Build Environ* 2015;88:97–105.
- [57] US Green Building Council. LEED v4 for building design and construction. Washington, D.C., USA: US Green Building Council; 2019.
- [58] Stanford Center for Biomedical Informatics Research. Protégé is a free, open-source ontology editor and framework for building intelligent systems. <https://protege.stanford.edu/>; 2020. accessed October 31, 2023.
- [59] Zhang Z, Lu B. Improving NSGA-II by a local search strategy with Gaussian mutation. In: 2021 40th Chinese control conference (CCC). Shanghai, China: IEEE; 2021. p. 1628–33.
- [60] Jin Y. Surrogate-assisted evolutionary computation: recent advances and future challenges. *Swarm Evolution Comput* 2011;1:61–70. <https://doi.org/10.1016/j.swevo.2011.05.001>.